업무자동화 필수 학습 정보 · AI 활용

AI는 어떻게 일을 대신해 주는가

처음부터 전부 외우는 자료가 아닙니다.
업무를 설명하고, 자동화하고, 검증할 때 필요한 핵심 정보를
순서대로 찾고 바로 적용할 수 있도록 구성했습니다.

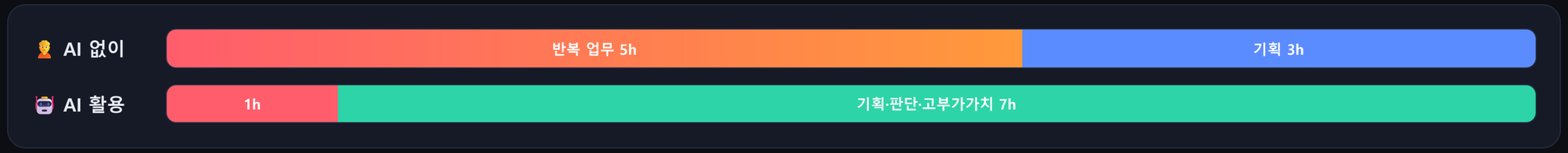
→ 또는 스페이스바 / 클릭으로 넘어갑니다 · 기존 94장 + 최신 실습 12장

@MOYI · ASTRA / 2026.09

WHY 왜 배우나

왜 이걸 배워야 하나?

같은 8시간이라도, AI를 다룰 줄 알면 **단순 반복 업무 시간은 줄이고, 기획과 판단에 쓰는 시간을 늘릴 수 있습니다.**



시간 단축

반복 자동화로 시간 되찾기

지식 확장

역질문과 역제안을 통해 한계를 넘어서기

품질 향상

명확한 지시로 정확한 결과

사고 예방

환각, 기밀 유출, 저작권 침해를 막는 기본기

> 내 시간을 **"대체 가능한 일"에서 "나만 할 수 있는 일"로 옮기는 것** — 이것이 이 학습 가이드의 목적입니다.

★ AI 시대 흐름

AI는 어떻게 발전해 왔을까요?

'빠른 답변'에서 '검증, 재시도, 도구 사용, 자율 작업'으로 진화했습니다. 🔥 표시는 실무 적용 전에 반드시 이해할 핵심 내용입니다.

~2019
룰 기반 챗봇
정해진 답변만 제공

2020
GPT-3 대형 LLM
빠른 답변에 초점

2022.11
ChatGPT 대중화
2개월 만에 1억 명

2023 초
🔥 **프롬프트 엔지니어링**
지시의 품질이 결과물의 수준을 결정함

2023
🔥 **RAG (외부 지식)**
외부 문서를 검색해 근거 기반으로 답변 (환각 현상 감소)

2024
🔥 **자기 검증·재시도**
빠른 답변에서 정확한 답변으로

2024-25
MCP 표준 + 추론 o1
답변하기 전에 스스로 '추론(생각)' 과정을 거침

2025
🔥 **하네스 + 코딩 에이전트**
다단계 자율 작업 수행

2025-26
멀티에이전트
장시간 자율 수행 및 AI 간 협업

> 빠른 답변 → 지시(프롬프트) → 외부 지식 활용(RAG) → 도구 사용(에이전트) → 검증 및 재시도(Reflection) → 자율 작업(하네스) → 협업으로 발전해 왔습니다.

START HERE 같은 강의라도 시작 과제는 다릅니다

먼저 내가 해결할 업무 하나를 고릅니다

설문은 실력을 판정하는 시험이 아닙니다. 관심 분야와 준비된 자료를 확인해 첫 실습의 난도를 정하는 출발점입니다.

관심 분야

반복업무, 문서·보고서, 데이터, 제작 중 어디에서 시작할지 고릅니다.

샘플 자료

공유 가능한 파일 한 개가 있는지 확인합니다.

첫 결과

표, 초안, 분석, 화면처럼 바로 확인할 결과를 정합니다.

검증 기준

누락, 형식, 계산, 사람이 확인할 지점을 적습니다.

나에게 필요한 AI 스터디 찾기

응답 결과를 바탕으로 첫 과제와 4주 흐름을 정합니다.

설문 열기 ↗

4 WEEKS 한 번 만들고 끝내지 않는 학습 경로

4주 동안 만들고, 틀려 보고, 다시 씁니다

① **좋은 질문**
업무 5줄과 완료 기준을 만듭니다.

② **업무 적용**
작은 샘플로 첫 결과를 만듭니다.

③ **예외 검토**
누락·오류·사람 확인을 찾습니다.

④ **반복 사용**
규칙과 결과를 남겨 다음에도 씁니다.

매주 남길 것

작동한 결과 하나, 확인한 실패 하나, 다음에 그대로 쓸 규칙 하나를 남깁니다.

성과를 보는 법

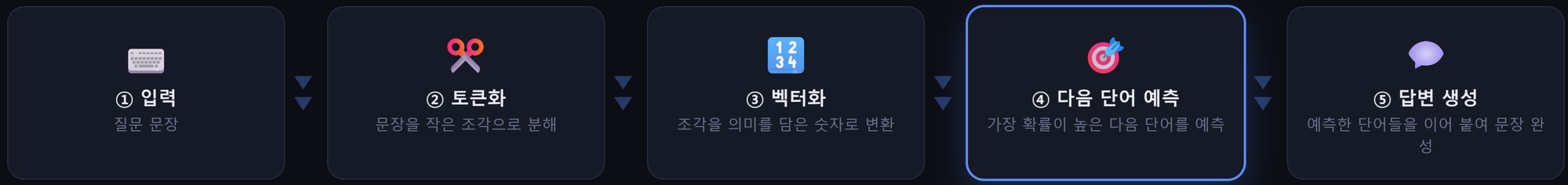
첫 답변 속도만 보지 않습니다. 검토와 재작업까지 포함한 전체 시간을 비교합니다.

> **다음 행동.** 오늘은 설문 결과와 상관없이, 공유 가능한 자료 한 개와 첫 결과물 한 개를 정하면 충분합니다.

01 LLM의 구조

AI는 '다음 단어 예측' 기계입니다

AI는 한 단어씩 확률을 계산해 가장 적절한 단어를 골라 문장을 만듭니다. 이 확률은 사용자가 제공한 맥락에 따라 달라집니다.



→ 실시간 생성 (한 단어씩):

> AI 모델들은 '다음 단어 예측'이라는 동일한 원리로 작동하지만, 학습한 데이터나 특화된 분야, 성격이 다르기 때문에 모델마다 다른 답변을 내놓습니다.

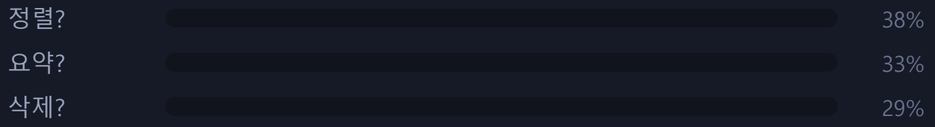
01 LLM의 구조

입력이 정확해야 출력이 정확하다

결과의 정확도는 사용자가 입력한 맥락에 달려 있습니다. **모호한 입력은 모호한 출력(또는 환각)을 만듭니다.**

❌ 흐릿한 입력

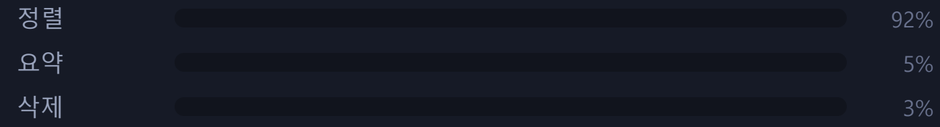
"그거 좀 정리해줘"



→ AI가 임의로 판단하여 틀리거나 환각을 일으킬 위험이 큽니다.

✅ 정확한 입력

"매출.xlsx B열 날짜를 오름차순 정렬"



→ 사용자의 의도대로 정확하게 수행합니다.

> 정확한 답변을 위해 제공해야 할 3가지 요소: 📍 데이터 위치, 🔗 소스 및 출처, 🧭 명확한 방향(목표)

01 LLM 내부 ①

토큰화 → 임베딩 → 의미 공간

문장을 조각(토큰)으로 나누고, 각 조각을 의미를 담은 좌표(벡터)로 변환합니다. 이때 비슷한 의미를 가진 단어들은 가까운 위치에 모이게 됩니다.

① 토큰화 — 텍스트를 조각내고 고유 번호 부여

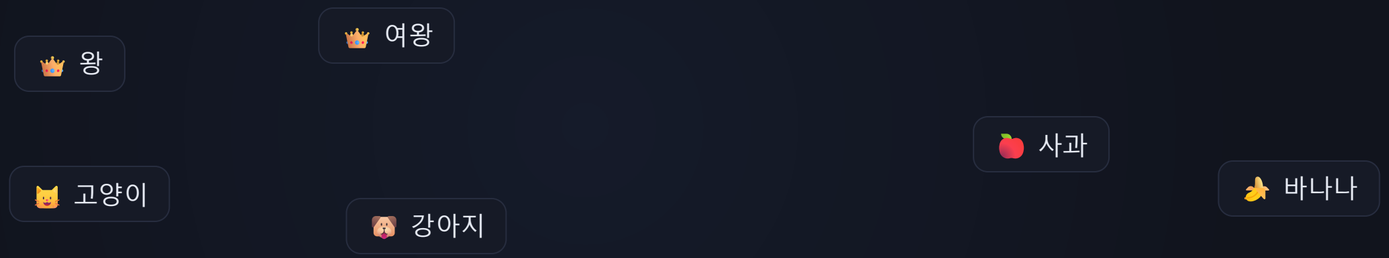
인공	지능	은	똑똑	하다
#8123	#4502	#11	#9981	#205

"인공지능은 똑똑하다" → ID 배열 [8123, 4502, 11, 9981, 205]

② 임베딩 — 각 토큰을 의미를 가진 벡터로 변환

지능 #4502 ▶ 

실제로는 768개에서 수천 개의 숫자로 이루어져 있으며, 이 숫자들의 조합이 곧 단어의 '의미 좌표'가 됩니다.



> '왕'과 '여왕'은 가까운 곳에, '사과'와 '바나나'는 다른 곳에 모입니다. 이를 통해 AI는 단어 간의 유사성을 이해하며, '왕 - 남자 + 여자 = 여왕' 같은 의미 연산도 가능해집니다.

01 LLM 내부 ②

위치 → 어텐션 → 층 → 출력

④ 어텐션(Attention) — 문맥상 어디에 주목해야 할지 결정하는 핵심 기술

색이 밝을수록 더 주목한다는 뜻입니다. 예를 들어 '찾았다'라는 단어는 '돈을'과 '은행'이라는 단어에 주목하여 문맥을 파악합니다.



⑤ 트랜스포머 층

여러 층을 거치며 단어의 의미와 문맥이 점점 정교하게 분석됩니다.

⑥ 출력 (Softmax)

다음에 올 단어들의 확률을 계산하여 가장 적절한 단어를 선택합니다.

똑똑하다	64%
유용하다	21%
빠르다	11%

③ 위치 인코딩 — 단어의 순서 정보 기억

"개가 pos1 사람을 pos2 물다 pos3"와 "사람이 개를 물다"는 단어는 같지만 위치가 달라 의미가 정반대입니다. 그래서 AI는 각 단어 조각(토큰)에 순서 정보를 함께 입력받습니다.

> ④ 트랜스포머 구조의 핵심인 '어텐션'은 특정 단어가 문장 내의 다른 단어들과 얼마나 연관되어 있는지 계산하여 전체적인 문맥을 이해하게 해줍니다.

01 LLM 학습

같은 질문, 학습 단계별로 답이 변한다

입력: "오늘 기분이 어때?" — 학습 단계별로 AI의 답변이 어떻게 발전하는지 살펴봅니다.

① 사전학습 직후

📚 방대한 인터넷 문서로 '다음 단어 예측'만 학습한 상태

🤖 *"오늘 기분이 어때? 그래, 가끔 사람들은 자기 기분에 대해 묻곤 하지. 옛날 그리스 철학자들은..."*

⚠️ 사용자의 질문에 답변한다는 개념이 없고, 단순히 입력된 문장에 이어지는 글을 작성합니다.

② 지도 미세조정(SFT) 후

👩‍🏫 사람이 작성한 좋은 질문과 답변 예시를 학습하여 모방하는 단계

🤖 *"안녕하세요. 저는 AI 언어 모델이므로 감정을 가지고 있지 않습니다. 도움이 필요하시면 알려주세요."*

✓ 답변하는 형식은 갖췄지만, 어투가 딱딱하고 사무적입니다.

③ 인간 피드백(RLHF) 후

👍 사람이 긍정적으로 평가한 답변 스타일을 학습하여 강화하는 단계

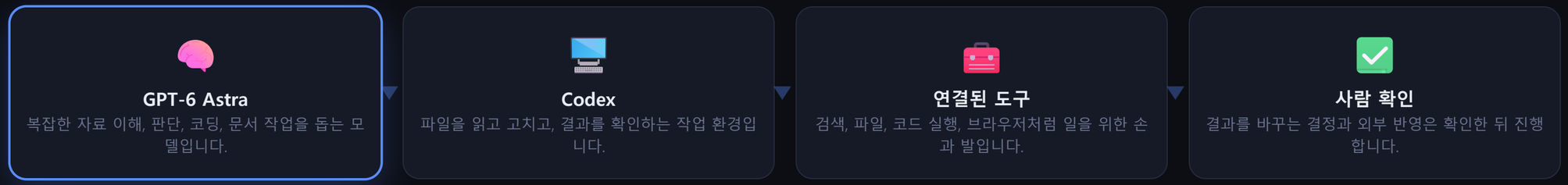
🤖 *"고마워요, 그렇게 물어봐 주셔서 😊 저는 감정은 없지만 오늘도 도움이 될 준비는 됐어요! 본인 기분은 어떠세요?"*

⭐ 말투가 자연스럽고 친절해지며, 사용자에게 되묻기도 합니다. 우리가 현재 사용하는 AI 모델들은 대부분 이 3단계까지 학습을 마친 상태입니다.

01 Astra 기준 작업 구조

Astra는 판단하고, Codex는 일을 끝낼 환경을 만듭니다

모델 하나를 고르는 일과 실제로 파일·코드·도구를 쓰는 환경은 다릅니다. 연결 범위가 있어야 결과까지 이어집니다.



계정에서 먼저 볼 것

선택 가능한 모델, 연결된 도구, 실행 권한과 현재 상태를 화면에서 확인합니다.

강의에서 지킬 말

모델 기능이 모든 계정의 도구 사용을 보장하지는 않습니다. 내 화면의 제공 범위를 우선 확인합니다.

공식 근거: GPT-6 Astra 모델 문서 · Astra 사용 가이드 · 기능과 제공 범위는 실습 직전에 각 계정에서 다시 확인합니다.

ASK WELL Astra와 함께 일하는 요청법

목적은 분명하게, 결과를 바꾸는 결정은 확인하게 합니다

목적
무엇을 해결할지 한 문장으로 씁니다.

입력·출력
쓸 자료와 남겨야 할 결과를 정합니다.

허용 범위
스스로 정할 일과 물어볼 일을 나눕니다.

검증
누락·형식·계산·사람 확인 기준을 적습니다.

```
목적: [해결할 업무]
입력: [파일·자료]
결과: [확인 가능한 산출물]
범위: 사소한 형식 결정은 진행하고, 결과·외부 반영을 바꾸는 결정은 먼저 알려줘.
검증: [누락·형식·계산·사람 확인 기준]을 확인한 뒤 결과와 미확인 항목을 구분해줘.
```

모델 차이는 왜 생기고, 내 업무에는 무엇이 맞을까요?

성능 차이를 만드는 4가지 요소

학습 데이터
양과 질

모델 크기
파라미터 수

훈련 방법
인간 피드백 기반 강화학습(RLHF) 및 정렬(Alignment) 방식

특화 튜닝
코드·검색 등

제품명보다 같은 샘플의 결과를 비교합니다.

1. 같은 입력
계약서, 보고서, 코드처럼 실제 업무와 비슷한 자료를 준비합니다.

2. 도구·권한
파일, 검색, 브라우저, 연결 서비스가 내 계정에서 가능한지 확인합니다.

3. 검증 결과
정확도·속도·비용·실행 성공률·검토 시간을 비교해 선택합니다.

> **Astra 기준 실습.** 먼저 Astra와 현재 연결된 도구로 작은 샘플을 끝까지 처리해 봅니다. 다른 모델은 같은 기준으로 비교한 뒤 선택합니다.

어떻게 지시하느냐에 따라 AI의 결과물이 완전히 달라집니다.

성공적인 프롬프트를 위한 5가지 요소 — 신입사원에게 업무를 지시하듯 구체적으로 설명해야 합니다.

역할
"너는 회계 담당자야"

맥락
"3분기 매출 보고서야"

구체적 지시
"중복 제거 후 합계"

출력 형식
"표 형태로 작성해 줘 / 3줄로 요약해 줘"

예시
"이런 식으로: ..."

❌ 모호한 지시

🙋 보고서 써줘

🤖 어떤 주제, 길이, 형식으로 써야 할지 몰라 되묻거나 임의로 작성하게 되어 여러 번 다시 시켜야 합니다.

✅ 명확한 지시

🙋 3분기 매출 보고서를 A4 1장 분량으로 써줘. 주요 데이터는 표로 정리하고, 매출 증감 이유를 3가지 포함해 줘.

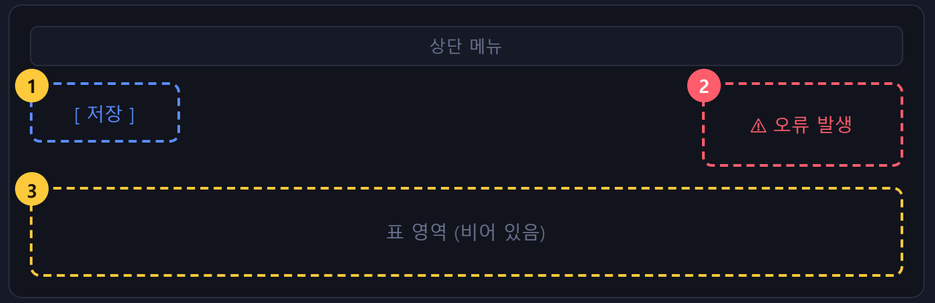
🤖 명확한 지시 덕분에 한 번에 정확한 결과물을 만들어냅니다. ✓

03 화면 캡쳐 + 주석

말로 길게 설명하는 대신, 화면을 캡처해서 문제가 있는 '위치'를 정확히 짚어주세요.

AI는 내 컴퓨터 화면을 볼 수 없습니다. 따라서 스크린샷을 찍고 박스나 번호로 핵심을 표시해 주면 AI의 이해도가 크게 높아집니다.

🖥️ 내 화면 + 주석



말로만 vs 캡쳐+주석

❌ '버튼을 눌렀는데 안 돼요.'라고 하면, AI는 '어떤 버튼인지' 몰라 계속 추측성 답변만 내놓습니다.

✅ 📷 화면 캡처에 번호를 매겨 '①저장 버튼 클릭 → ②오류 발생 → ③표가 비어 있음'을 보여주면, '②번 오류는 날짜 형식 때문이니 이렇게 고치세요.'라고 정확히 답해 줍니다.

백 번 설명하기보다 한 번 보여주는 것이 낫습니다.

04 파이썬 엑셀

수작업으로 1,000번 클릭해야 할 일을 코드 몇 줄로 단 0.2초 만에 처리합니다.

파이썬은 엑셀 파일을 '열고, 데이터를 처리하고, 다시 저장하는' 과정을 빠르고 지치지 않게 수행하는 든든한 일꾼입니다.

사람이 손으로

파일 50개를 일일이 열어서 정렬하고, 중복 데이터를 제거한 뒤 저장하는 반복 작업

3시간
파일 50개

실수 발생 확률 높음
데이터 누락이나 오타 발생

VS

파이썬 코드로

```
import pandas as pd df = pd.read_excel("매출.xlsx") df = df.drop_duplicates().sort_values("날짜") df.to_excel("정리완료.xlsx")
```

0.2초
파일 50개도 반복문 하나로 순식간에 처리

정확
매번 동일하고 정확하게 작업 수행

05 OCR → 엑셀

OCR 정확도를 끌어올리는 실전 기법

인식할 영역(ROI) 지정 + 이미지 확대 및 보정 + 여러 OCR 엔진 결과를 비교하는 투표 방식 + 사람의 최종 검증



① ROI 지정

인식할 핵심 영역만 지정하여 불필요한 노이즈를 제거합니다.



② 확대·전처리

이미지 해상도를 높이고 명암 대비를 조절해 글자를 선명하게 만듭니다.



③ 여러 OCR 동시

Tesseract, PaddleOCR, AI 비전 모델 등 여러 엔진을 동시에 사용합니다.



④ 투표 확정

여러 엔진의 결과 중 가장 많이 일치하는 값을 다수결로 확정합니다.



⑤ 불일치→사람

엔진 간 결과가 다를 때만 사람이 직접 개입하여 검토합니다(Human-in-the-loop).

다중 OCR 투표 방식 예시 — 청구서의 '수량' 칸을 읽을 때

Tesseract **120**

EasyOCR **120**

PaddleOCR **12O**

AI(멀티모달) **120**

✓ 4개의 엔진 중 3개가 '120'으로 동일하게 인식했으므로 이를 최종 결과로 확정합니다.

06 API

API란 프로그램끼리 데이터를 주고받기 위해 미리 약속해 둔 '창구'입니다.

식당의 웨이터와 같습니다. 손님은 주방의 요리 과정을 몰라도, 웨이터에게 정해진 메뉴를 주문하면 완성된 요리를 받을 수 있습니다.



이러한 API의 원리로 자동화 시스템이 구축됩니다.

1. 데이터 읽기
2. API로 AI 요청
3. AI가 답 반환
4. 엑셀에 자동 입력
5. 1,000행 반복

06 API 데이터 흐름

데이터가 시스템 사이에서 어떻게 전송되고, 저장, 수정, 반환되는지 알아봅시다.

연락처 등록을 예로 들어보겠습니다. 데이터는 주로 JSON이라는 정형화된 텍스트 형식으로 오고 갑니다.

🙋 📩 POST /api/contacts

```
{
  "name": "김철수",
  "phone": "010-1234-5678"
}
```

💾 DB → INSERT

```
contacts 테이블에 새로운 행으로 저장
  id = 1042
  name = "김철수"
  phone = "010-1234-5678"
```

🙋 🖍️ PUT /api/contacts/1042

```
{ "phone": "010-9999-1111" }
```

🤖 📩 GET → 200 OK

```
{
  "id": 1042,
  "name": "김철수",
  "phone": "010-9999-1111",
  "updated": "2026-06-03 14:25"
}
```

> 데이터 생성(POST), 수정(PUT), 조회(GET) 등의 요청이 처리됩니다. 이처럼 모든 데이터가 JSON 형식으로 통신되기 때문에 웹이든 앱이든 동일한 데이터를 다룰 수 있으며, 이것이 곧 자동화의 토대가 됩니다.

06 API 일상

API를 활용하면 우리의 일상이 이렇게 편리해집니다.

매일 직접 검색해서 확인하던 정보들을, 이제는 시스템이 알아서 내게 가져다주게 됩니다.

🙋 오늘 미세먼지 어때?

🤖 (기상청 API 활용) 오늘은 '나쁨' 수준입니다. 😷 마스크를 챙기세요.

🌫️ 미세먼지·날씨

🙋 153번 버스 언제 와?

🤖 (버스 API 활용) 153번 버스가 3분 후 도착 예정입니다. 🚌

🚌 버스 도착

🙋 우리 동네 시세 올랐어?

🤖 (국토교통부 실거래가 API 활용) 지난달 대비 시세가 1.2% 상승했습니다. 📈

🏠 부동산 실거래가

🙋 이번 달 외식비 얼마 썼어?

🤖 (카드사 API 활용) 이번 달 외식비 지출은 32만 원입니다. (결제 내역 자동 분류)

💳 자동 가계부

🙋 도쿄 항공권 싸지면 알려줘

🤖 (항공권 예약 API 활용) 도쿄행 항공권 가격이 18만 원으로 떨어졌습니다! ✈️ (알림 발송)

✈️ 항공권 추적

🙋 오늘 AI 뉴스 요약해줘

🤖 (뉴스 API 활용) 오늘 주요 AI 뉴스는 총 3건입니다: ①신규 모델 발표, ②AI 규제 동향, ③기업 투자 소식.

📰 뉴스 요약

07 터미널

터미널(명령 프롬프트) 환경이 강력한 이유

마우스로 아이콘을 클릭하는 방식(GUI) 대신, 직접 명령어를 입력하는 방식(CLI)을 쓰면 작업을 훨씬 빠르고 정확하게 자동화할 수 있습니다.



① 직접·정확

텍스트로 명확하게 명령을 내리므로, 모호함 없이 의도한 대로 정확히 실행됩니다.



② 반복을 한 줄로

파일 100개를 처리하는 일도 명령어 한 줄이면 충분합니다.



③ 저장→재실행

자주 쓰는 명령어들을 스크립트 파일로 저장해 두면 언제든 버튼 한 번으로 다시 실행할 수 있습니다.



④ 연결(파이프)

하나의 작업 결과를 다음 작업의 입력으로 바로 연결하여(파이프) 연속 처리할 수 있습니다.

```
$ rename 사진_*.jpg → 행사_*.jpg
  ✓ 100개의 파일 이름을 0.1초 만에 일괄 변경 완료
$ python 정리.py   # 엑셀 50개 자동 처리
  ✓ 50개의 엑셀 파일 처리 후 '정리완료' 폴더에 일괄 저장 완료
```

08 막힘 돌파

작업 중 막혔을 때는? — 검증된 외부 자원 탐색과 AI의 역제안 활용하기

문제 해결 5단계 사이클 — 완벽하게 한 번에 풀려 하지 말고, 이 과정을 빠르게 반복하는 것이 중요합니다.



① 막힘 포착

발생한 에러 메시지나 상황을 캡처하여 확인합니다.



② 자원 탐색

GitHub에서 검증된 오픈소스나 시스템(MCP) 문서를 찾아봅니다.



③ AI 역제안 요청

AI에게 '이 문제를 해결할 더 나은 방법이 있을까?'라고 역제안을 요청합니다.



④ 적용

AI가 제안한 해결책을 실제 코드나 작업에 적용해 봅니다.



⑤ 검증 & 반복

테스트 후 여전히 문제가 있다면 다시 ①번 단계로 돌아갑니다.

↺ 막힐 때마다 고민을 길게 하지 말고, 다시 ①번 단계로 돌아가 이 문제 해결 사이클을 빠르게 반복하세요.

08 역제안·지식 확장

AI에게 역질문과 역제안을 요청하면 내가 가진 지식의 한계를 뛰어넘을 수 있습니다.

마케팅 예시

🙋 아이폰은 주로 여성이 쓰니까 여성 타겟이지?

🤖 그 타겟 설정은 다소 좁습니다. 아이폰은 전 세대와 다양한 직군에서 사용합니다. 마케팅 방향으로 ①업무 생산성 향상, ②Z세대의 트렌디한 감성, ③가족 간의 연결, ④프리미엄 이미지 등 4가지를 제안합니다.

🙋 ② Z세대 감성으로!

🤖 Z세대를 겨냥한다면 인스타그램과 틱톡을 중심으로 한 숏폼 영상 챌린지, 트렌디한 컬러 키워드 마케팅, 그리고 또래 인플루언서와의 협업을 추천합니다.

A로 시작하는 단어

🙋 내가 떠올릴 수 있는 'A'로 시작하는 마케팅 관련 단어:

Apple

🤖 방대한 데이터를 가진 AI가 제안할 수 있는 'A' 관련 마케팅 개념들:

Apple Amazon Audience Analytics A/B Awareness Acquisition Algorithm Affiliate Ad ...

이처럼 AI는 내가 미처 생각하지 못한 지식의 빈칸을 훌륭하게 채워줍니다.

08 구조 학습

계속해서 코드를 '다시 만드는' 시행착오를 줄이려면 — 개발 전 먼저 '구조'부터 학습해야 합니다.

머릿속에 있는 아이디어만으로 무작정 코드를 짜라고 지시하면, 기초 설계가 부실해져 결국 처음부터 다시 만들어야 하는 상황이 생깁니다.

❌ 바로 시키기

🙋 "예약 시스템 만들어줘"

🤖 (AI가 뼈대 없이 임의로 구현함) → 에러 발생 → 코드 수정 → 결제 기능 추가 실패 → 또 수정 → 알림 기능 연동 실패 → 끝없는 재작업...

→ 기초 토대가 부실하여 기능이 추가될 때마다 코드를 완전히 새로 짜야 합니다.

✅ 구조 먼저 학습 후 구현

🙋 '예약 시스템을 만들려면 어떤 기능들이 필요하고, 전체적인 데이터 흐름과 시스템 구조는 어떻게 잡아야 할까?'

🤖 시스템 구성 요소: ①사용자 인증, ②예약 데이터베이스, ③결제 시스템, ④알림 전송, ⑤관리자 페이지. 전체 흐름: 예약 요청 → 빈자리 확인 → 결제 진행 → 예약 확정 → 안내 알림 발송.

→ 이렇게 탄탄한 토대를 먼저 세우고 나면, 필요한 부분만 수정하거나 기능을 쉽게 확장할 수 있습니다.

> '학습 → 설계 → 구현' 순서로 진행하면 불필요한 재작업 굴레에서 벗어날 수 있습니다. AI는 코딩 도구일 뿐만 아니라, 시스템 구조를 가르쳐주는 훌륭한 선생님이기도 합니다.

AI가 자꾸 같은 실수를 반복할 때 해결하는 6가지 방법

🙋 또 그 실수네... 하지 말랬잖아

🤖 (대화 세션이 바뀌면 이전의 주의사항을 잊어버리는 AI의 휘발성 기억력)

🤖 프로젝트 내에 CLAUDE.md 같은 규칙 파일을 만들어두면, 매 세션마다 AI가 이를 읽고 자동으로 규정을 준수합니다.
(매번 말로 지시하는 대신, 시스템적인 규칙과 검증 구조를 만드세요.)



① MD 규칙 파일

CLAUDE.md나 AGENTS.md 같은 마크다운 문서에 프로젝트 규칙을 명시해 매 세션 자동 적용



② 스키마 재설계

데이터 구조나 로직이 모호해서 오류가 잦다면, 구조 자체를 단순하고 명확하게 재설계하여 실수 가능성 차단



③ 출력 형식 강제

JSON 스키마 등을 활용해 정해진 출력 형식에 맞지 않으면 AI의 응답을 시스템이 자동 거부하도록 설정



④ 테스트 가드

검증용 테스트 코드를 만들고, 이를 통과하지 못하면 AI가 임의로 작업을 완료했다고 선언하지 못하게 강제



⑤ 훅 사전 차단

파일 삭제나 운영 서버 배포 같은 위험한 동작은 시스템 훅(Hook)을 통해 AI가 임의로 실행하지 못하게 사전 차단



⑥ 분해 + 메모리

복잡한 작업은 작게 분해하여 하나씩 지시하고, 진행 상황은 MEMORY.md 파일에 누적 기록하여 문맥 유지

TIP 완료 조건부터

무슨 기능을 만들지보다 '어떤 상태가 되어야 작업이 완료된 것인지'를 먼저 명확히 정의해야 합니다.

❌ 기능만 요청

품목을 매칭해줘.

비슷한 품목끼리 매칭했습니다.

정확도는 어느 정도여야 하는지, 예외 상황은 어떻게 처리할지, 검증은 어떻게 할지에 대한 기준이 전혀 없습니다.

✅ 완료 조건까지 요청

- 코드가 같으면 우선 매칭
- 90% 이상 자동 확정
- 나머지는 검토 목록
- 원본·결과 행 수 일치

이처럼 개발을 시작하기 전에 AI에게 '이 작업의 완료 조건과 검증 방법을 먼저 정리해서 알려줘'라고 지시하세요.

TIP 정상·경계·실패

AI에게 잘 처리된 성공 사례만 알려주지 말고 의도적으로 실패 처리해야 하는 사례의 예시도 함께 제공해야 합니다.

✅

정상 사례

프리미엄 원두 A · 1kg
→ 원두 프리미엄 A · 1kg

자동 확정

⚠️

애매한 사례

종이컵 대 · 16oz
→ 종이컵 16온스

이런 경우는 임의로 처리하지 말고 사람이 직접 검토하도록 보류

⛔

실패 사례

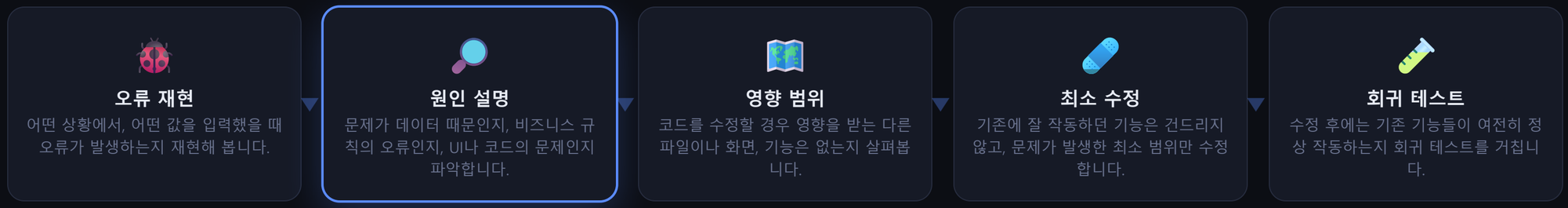
시즌 블렌드 · 규격 없음
→ 자동 매칭 금지

규격 정보가 없어 완전히 매칭 실패로 분류

> 💡 **프롬프트 추천 문구:** '정상적으로 처리되어야 할 사례, 애매해서 보류해야 할 사례, 그리고 아예 실패로 처리해야 할 사례를 각각 테스트해 줘. 만약 예상과 다른 결과가 나온다면 작업을 멈추고 그 원인을 먼저 설명해 줘.'

TIP 오류 수정 순서

오류가 났을 때 AI에게 바로 코드를 고치라고 지시하지 말고 문제의 정확한 원인과 코드를 수정했을 때 영향을 받는 범위를 먼저 확인해야 합니다.



💡 프롬프트 추천 문구: '코드를 바로 수정하지 말고 다음 세 가지를 먼저 알려줘. 1. 오류의 발생 원인, 2. 변경해야 할 파일 목록, 3. 이번 수정으로 인해 영향을 받는 다른 기능들. 이 내용을 확인한 후, 기존의 정상 기능은 그대로 유지하면서 최소한의 범위 내에서 코드를 수정해 줘.'

TIP 추측 대신 질문

AI가 요구사항을 명확히 모를 때는 임의로 추측하지 말고 사용자에게 질문하도록 지시하고, 질문과 함께 AI가 생각하는 추천 설정값(기본값)과 그 이유도 함께 제시하도록 만듭니다.

requirement-interview

사용자

🙋 '재고 부족 알림 기능을 시스템에 추가해 줘. 개발에 필요한 정보가 부족하다면 임의로 가정해서 만들지 말고, 내게 질문을 해. 단, 한 번에 너무 많은 질문을 하지 말고 최대 3개까지만 물어봐 줘.'

AI

① 부족 기준은 0개인가요, 안전재고 이하인가요?
② 알림은 이메일인가요, 메신저인가요?
③ 외부 발송 전 승인이 필요한가요?

추천 기본값: 추천 설정: 알림 기준은 '안전재고 이하', 알림 방식은 '사내 메신저 초안 작성', 발송 방식은 '담당자 승인 후 외부 발송'
추천 이유: 이렇게 설정하면 재고 확인 누락을 줄이면서도, 잘못된 내용이 외부 거래처로 잘못 발송되는 사고를 막을 수 있습니다.

💡 **프롬프트 작성 요령:** *AI에게 단순히 '모르면 질문해'라고만 하면, 전문 지식이 부족한 사용자는 질문을 받아도 어떻게 대답해야 할지 막막할 수 있습니다. 반드시 '모르면 네가 생각하는 추천안과 그 장단점을 함께 제시해 줘'라는 조건을 덧붙이세요.*

TIP 코드와 AI의 분업

정해진 계산은 코드가, 뜻을 이해하고 문장을 만드는 일은 AI가 맡습니다

⚙ 일반 코드에게

- 열 이름 변경
- 합계·증감률 계산
- 품목 코드 완전 일치
- 중복·누락 검사

일반 코드는 처리 속도가 빠르고 비용이 거의 들지 않으며, 동일한 입력에 대해 항상 100% 동일한 결과를 보장합니다.

🧠 AI에게

- 유사한 품목명 판단
- 회의·보고 내용 요약
- 특이사항과 원인 설명
- 보고서 문장 생성

AI는 텍스트의 숨은 의미나 복잡한 맥락 파악이 필요한 영역에만 선택적으로 사용하는 것이 효율적입니다.

AI API 호출량 **(꼭 필요한 곳에만 사용하므로)** 비용 ↓ 속도 ↑ 재현성 ↑

CONTEXT 대화가 길어질 때 실제로 일어나는 일

컨텍스트 압축과 새 세션은 같은 초기화가 아닙니다

현재 대화

요구사항·수정 이유·실패 기록과 최근 작업이 함께 들어 있습니다.

세부 맥락이 가장 풍부함

컨텍스트 압축

긴 대화를 요약해 핵심을 남기고 사용 가능한 공간을 확보합니다.

핵심은 이어지지만 세부 내용은 빠질 수 있음

클리어·새 세션

채팅 컨텍스트를 새로 시작합니다. 이전 대화를 자동으로 안다고 기대하면 안 됩니다.

파일과 문서로 남긴 정보부터 다시 읽음

> **정확한 표현.** 압축은 완전한 기억 삭제가 아니라 **손실이 있을 수 있는 요약**입니다. 새 세션은 새로운 담당자에게 업무를 넘기는 것에 더 가깝습니다.

MODEL EVOLUTION 강해진 모델이 보완한 부분

최근 모델은 자료를 더 오래 붙들고, 작업을 마무리하는 능력을 넓혔습니다

OpenAI · GPT-6 Astra

2026-09-09 강의 기준의 모델 참고 사례입니다. 복잡한 자료 이해, 추론, 코딩, 컴퓨터 사용과 문서 작업을 연결해 봅니다.

내 계정의 모델·도구·권한을 먼저 확인

Anthropic · Sonnet 5 · Opus 4.8 · Fable 5

Sonnet 5는 계획, 도구 사용, 코딩과 장기 작업 지속성이 강화됐고, 이전에 더 큰 모델이 필요했던 작업을 더 효율적으로 수행하도록 발전했습니다.

더 긴 집중 + 자기검증 + 다단계 작업 완수

📚 긴 자료 이해

더 많은 코드와 문서를 한 번에 참고

🧭 계획 유지

여러 단계의 목표와 순서를 더 안정적으로 추적

🧰 도구 사용

검색·터미널·파일·API를 연결해 실제 행동

✅ 자기검증

테스트와 결과 확인을 스스로 수행할 가능성 증가

공식 근거: GPT-6 Astra 모델 문서 · 모델명·사양·제공 범위는 자료 활용 시점에 다시 확인합니다.

PLATFORM TIMELINE 언제, 무엇을 할 수 있게 됐나

AI는 대화창을 넘어 문서·브라우저·앱·전화에서 직접 일하기 시작했습니다

OpenAI ChatGPT	**2025** Deep research·Apps·Codex 조사·연결·코딩으로 확장	**2026.06** Memory Dreaming·File Library 과거 맥락과 파일을 다시 활용	**2026.07 참고 사례** GPT-5.6·ChatGPT Work 당시 공개된 긴 업무 실행 기능
Anthropic Claude	**2024~25** Artifacts·Claude Code·MCP 결과물·코드·외부 도구 연결	**2026.03** Cowork·컴퓨터 사용·시각화 PC 작업과 차트를 직접 수행	**2026.06** Sonnet 5·Claude Tag 에이전트 작업과 팀 위임 강화
Google Gemini	**2024~25** NotebookLM·Workspace·Flow 자료·업무 앱·미디어 연결	**2026.03** Docs·Sheets·Slides 협업 메일·파일 맥락으로 결과 제작	**2026.05~06** Spark·프로액티브 브리핑 컴퓨터 작업과 선제적 도움
xAI Grok	**2025** DeepSearch·Agent Tools 웹·X·코드 도구 연결	**2026.04~06** Voice·Imagine 1.5·Build 통화·영상·코딩·문서 제작	**2026.07** Grok 4.5·Automations 예약·메일 조건으로 자동 실행

> **2026년 8월 참고 사례.** 이 타임라인은 당시 공개 정보를 정리한 자료입니다. 최신 기능으로 단정하지 않으며, 내 계정에서 실제로 켜져 있는지 먼저 확인합니다.

공식 릴리스: OpenAI · Anthropic · Google · xAI (확인 2026-08-05)

PLATFORM MAP 기능별로 고르는 메인스트림 도구

AI마다 강점이 다르고, 일하는 화면과 도구도 달라졌습니다

업무 기능	OpenAI · Codex	Anthropic · Claude	Google · Gemini	xAI · Grok
대화·조사	ChatGPT · Deep research	Claude · Research	Gemini · Deep Research	Grok · DeepSearch · X 실시간 검색
코딩·앱 제작	Codex 앱 · CLI · IDE · 클라우드	Claude Code · 플러그인 · Skills	AI Studio · Firebase Studio · Gemini CLI	Grok Build · API · Kilo Code 연동
문서·오피스	파일 생성 · 앱 커넥터	Excel·PPT·Word·PDF 생성 / M365 추가 기능	Workspace의 Gemini · Gemini Notebook	문서 분석·생성 중심
시각·미디어	이미지 생성 · 앱 제작	Artifacts · 시각화 · 인터랙티브 결과물	Flow · 이미지·영상 · 오디오 개요	Grok Imagine 이미지·영상·음성 API
자동 실행	컴퓨터 사용 · 브라우저 · 자동화	Computer use · Claude Tag · MCP	Agent mode · Opal · Workspace 자동화	Agent Tools API · Voice Agent Builder

> **선택 기준.** "어느 모델이 1등인가?"보다 **내 자료가 어디에 있고, 결과물이 무엇이며, 어떤 도구를 실제로 조작해야 하는가?**를 먼저 묻습니다.

CHATGPT · OPENAI 대화에서 완성된 업무 결과물로

ChatGPT는 이제 질문 답변보다 긴 업무를 끝까지 수행하는 쪽으로 확장됐습니다

ChatGPT Work

끝까지 작업 — 조사하고, 연결된 앱과 파일을 읽고, 완성된 문서·표·발표·사이트를 만듭니다.

진행 중 개입 — 작업 과정을 보고 질문에 답하거나 방향을 바꿉니다.

중요 행동 승인 — 외부 변경처럼 책임이 따르는 단계는 사용자가 확인합니다.

예: 주간 자료 수집 → 분석 → 경영 보고서와 발표자료 완성

GPT-6 Astra · Codex · 작업 기록

Astra — 복잡한 판단과 구현을 연결하는 강의 기준 모델입니다.

Codex — 코드 저장소를 읽고 구현·테스트·검토하며 작업을 진행합니다.

AGENTS.md·CURRENT_TASK.md — 프로젝트 규칙과 현재 상태를 파일로 남깁니다.

주의: 실제 제공 모델과 연결 도구는 계정에서 다시 확인

> **추천 업무.** 자료가 여러 앱에 흩어져 있고, 조사부터 최종 파일까지 한 번에 맡기고 싶을 때 적합합니다.

공식 근거: GPT-6 Astra 모델 문서 · 과거 릴리스는 당시 참고 사례로만 봅니다.

CLAUDE · ANTHROPIC 문서·코드·컴퓨터를 한 작업으로

Claude는 생각한 결과를 문서와 화면에 직접 반영하는 기능이 강해졌습니다

Cowork · Computer use

화면 직접 조작 — 파일을 열고 개발 도구와 앱을 클릭해 작업합니다.

원격 지시 — 휴대전화의 지속 스레드로 데스크톱 작업을 이어서 관리합니다.

시각화 — 대화 안에 맞춤형 차트·도표·인터랙티브 결과를 만듭니다.

예: 폴더의 보고서 취합 → 표 생성 → 이상값 확인 → 초안 작성

Sonnet 5 · Office · Claude Tag

Sonnet 5 — 추론·도구 사용·코딩·전문 업무가 강화된 에이전트형 모델입니다.

Excel ↔ PowerPoint — 두 추가 기능이 대화 맥락을 공유해 분석과 발표를 이어갑니다.

Claude Tag — Slack에서 Claude를 불러 대화 맥락의 업무를 위임합니다.

주의: 컴퓨터 조작·외부 발송·파일 변경은 권한과 승인 범위를 먼저 제한

> **추천 업무.** 긴 문서와 코드를 함께 다루거나, Excel 분석에서 PowerPoint 보고까지 같은 맥락으로 이어갈 때 적합합니다.

출처: Claude Release Notes — 2026-03~06 · Anthropic Sonnet 5 발표

GEMINI · GOOGLE 내 자료와 Google 도구를 연결

Gemini는 질문을 기다리는 도구에서 먼저 준비하고 앱을 움직이는 비서로 확장됩니다

Gemini Spark · 프로액티브 브리핑

- **Spark** — 컴퓨터 화면과 로컬 앱을 사용해 여러 단계의 일을 수행하는 에이전트입니다.
- **Daily briefs** — 일정·메일·관심사를 바탕으로 필요한 내용을 먼저 준비합니다.
- **Gemini 3.5 Flash** — 빠른 응답과 행동을 함께 겨냥한 차세대 모델입니다.

예: 아침 일정·메일 요약 → 준비할 문서와 후속 작업 제안

Workspace · NotebookLM · 제작 도구

- **Docs·Sheets·Slides·Drive** — 메일·채팅·파일 맥락으로 초안·분석·발표를 만듭니다.
- **NotebookLM** — 지정한 자료에 근거해 답하고 출처·오디오·학습 결과물을 만듭니다.
- **AI Studio·Flow·Firebase Studio** — 모델 시험, 영상, 앱 제작으로 이어집니다.

주의: 개인용·업무용 계정과 Alpha·Labs 제공 범위가 서로 다름

> **추천 업무.** Gmail·Drive·Docs·Sheets에 업무자료가 쌓여 있고, 검색과 작성 사이의 이동을 줄이고 싶을 때 적합합니다.

출처: Google Gemini App I/O 2026 발표 · Google Workspace 2026-03 발표

MICROSOFT 365 COPILOT Office 문서 안에서 직접 실행

Copilot은 답변을 넘어 Word·Excel·PowerPoint를 직접 수정합니다

Word

초안·재작성·구조 변경·문체 조정을 문서 안에서 실행합니다.

근거와 인용 정확성 확인

Excel

수식·표·피벗·차트까지 분석하고 통합문서에 직접 반영합니다.

원본·수식·결과 검증

PowerPoint

회사 템플릿을 따르며 기존 발표를 새 데이터와 메시지로 업데이트합니다.

레이아웃·수치·브랜드 검수

Work IQ

메일·회의·문서 등 업무 신호를 이용해 현재 작업의 맥락을 이해합니다.

Agent 365

조직이 여러 에이전트의 권한·사용·보안 상태를 발견하고 관리합니다.

> **추천 업무.** 최종 결과가 Word·Excel·PowerPoint이고, 회사의 Microsoft 365 권한과 문서 체계를 그대로 사용해야 할 때 적합합니다.

출처: Microsoft 365 Blog — Copilot agentic capabilities GA 2026-04 · Agent 365 2026-05

GROK · xAI 실시간 조사·제작·예약 실행

Grok은 실시간 검색을 넘어 조건에 맞춰 스스로 다시 실행되는 작업을 지원합니다

Grok 4.5 · Build · Office 결과물

Grok 4.5 — 코딩·에이전트·지식업무를 겨냥한 최신 모델입니다.

Grok Build — 앱과 코드를 만들고 Excel 모델·PowerPoint·Word 결과물도 생성합니다.

DeepSearch·X — 웹 자료와 X의 실시간 흐름을 함께 조사합니다.

예: 시장 반응 조사 → 경쟁사 비교 → Excel·PPT 보고

Automations · Voice · Imagine

Automations — 매일·매주 같은 일정이나 새 이메일 도착을 조건으로 작업합니다.

Voice Agent — 전화에서 정보를 듣고 도구를 호출해 예약·상담을 처리합니다.

Imagine Video 1.5 — 움직임·물리·음성 동기화를 강화한 영상을 만듭니다.

주의: 자동 발송·계정 변경·결제는 사람 승인과 실행 한도 설정

> **추천 업무.** 실시간 여론·뉴스가 중요하거나, 반복 조사와 이메일 조건 자동화를 대화식으로 빠르게 만들고 싶을 때 적합합니다.

출처: xAI Grok 4.5 · Automations · Imagine Video 1.5 공식 발표 — 2026-06~07

WHERE AI WORKS CLI·IDE·브라우저·API·MCP 읽는 법

같은 모델도 어디에서 무엇과 연결해 쓰느냐에 따라 할 수 있는 일이 달라집니다

⌨️ CLI

명령어를 입력하는 터미널

파일 검색·프로그램 실행·테스트·Git 작업을 빠르게 이어갑니다.

뜻: 반복 개발 작업에 강함

🧩 IDE

코드를 쓰고 고치는 작업실

현재 파일·오류·프로젝트 구조를 보며 바로 수정합니다.

뜻: 사람과 AI가 함께 편집

🌐 브라우저

웹 화면을 보고 조작하는 공간

페이지를 읽고 클릭·입력·다운로드하며 결과를 확인합니다.

뜻: 실제 화면 업무까지 확장

🔗 API

서비스끼리 정해진 형식으로 통신

메일·ERP·결제·모델 기능을 코드에서 안정적으로 호출합니다.

뜻: 대량·반복 실행에 적합

🔌 MCP

AI용 도구 연결 규격

파일·DB·업무 앱의 기능과 사용법을 모델에 알려 줍니다.

뜻: 연결 방식을 재사용

🖥️ 컴퓨터 사용

화면 좌표와 시각 정보를 이용

API가 없는 오래된 프로그램도 사람처럼 조작합니다.

뜻: 범용적이지만 검증 필요

> **핵심.** CLI와 IDE는 개발 현장, 브라우저와 컴퓨터 사용은 화면 업무, API와 MCP는 서비스 연결 방식입니다. 제품명이 아니라 AI가 일하는 장소와 연결 방식을 뜻합니다.

AGENT LOGIC 기능이 달라도 엔진은 비슷합니다

에이전트 기능은 대부분 판단과 도구 실행을 반복하는 구조로 움직입니다



1. 지시 해석
목표·조건·완료 기준을 찾음

↓



2. 문맥 수집
파일·화면·개인 자료를 확인

↓



3. 계획
필요한 단계와 도구를 선택

↓



4. 실행
생성·검색·클릭·API 호출

↓



5. 관찰
도구가 돌려준 결과를 다시 읽음

결과가 부족하면

프롬프트·장면·검색어·도구 입력값을 바꿔 다시 실행합니다.

위험한 작업이면

결제·발송·삭제·공개 전에는 사람의 승인을 요청합니다.

완료 기준을 만족하면

파일·링크·작업 로그와 함께 결과를 전달합니다.

> **주의.** 화면에는 내부 사고 과정을 그대로 보여 주는 것이 아니라, 입력·도구 호출·관찰 결과·승인처럼 확인 가능한 실행 흔적을 보여 줍니다.

MEDIA LOGIC 이미지와 영상은 어떻게 만들어지나

“만들어 줘” 뒤에서는 장면을 구조화하고 생성 결과를 다시 검사합니다

🖼️ 이미지 생성

지시 “우리 제품 사진으로 여름 광고를 만들어 줘.”

해석 제품·배경·구도·문구·비율·금지 요소를 나눕니다.

생성 이미지 모델이 지시와 참고 이미지를 바탕으로 픽셀을 만듭니다.

검사 제품 형태·글자·로고·손가락·저작권 위험을 확인합니다.

수정 지시 → 필요한 영역만 편집하거나 다시 생성

🎬 영상 생성

지시 “15초 제품 소개 영상을 세 장면으로 만들어 줘.”

계획 장면·시간·카메라 움직임·대사·음향을 정합니다.

생성 영상 모델 또는 Remotion 같은 렌더러가 프레임을 만듭니다.

검사 인물·제품의 연속성, 자막, 길이, 장면 전환을 확인합니다.

장면별 수정 → 다시 렌더링 → 최종 파일 출력

> **차이.** 생성 모델은 픽셀과 프레임을 직접 만듭니다. Remotion은 LLM이 코드를 작성하고 렌더러가 영상으로 출력하므로 숫자·자막·레이아웃 수정이 더 쉽습니다.

참고: OpenAI 이미지 생성·Sora 공식 문서 · Remotion 공식 문서

BROWSER LOGIC Claude in Chrome이 일하는 방식

브라우저 에이전트는 답만 쓰지 않고 현재 페이지를 관찰하고 조작합니다

💁 거래처 사이트에 로그인해서 이번 달 주문 내역을 내려받고, 누락 주문만 알려 줘.



페이지 읽기
탭·버튼·표·입력란을 파악

↓



행동 계획
로그인·조회·다운로드 순서를 정함

↓



클릭·입력
허용된 화면 안에서 작업

↓



결과 수집
다운로드 파일과 화면 변화를 확인

↓



비교·보고
기준 자료와 대조해 예외만 전달

잘하는 일

웹 조사, 반복 입력, QA, 다운로드처럼 화면에서 확인 가능한 작업

사람이 승인할 일

로그인 정보 입력, 결제, 발송, 삭제, 게시처럼 되돌리기 어렵거나 책임이 따르는 작업

출처: Anthropic Claude in Chrome 시작·권한·안전 가이드

하나는 대화를 실시간 행동으로, 다른 하나는 내 자료를 맞춤 답변으로 바꿉니다

☎ Voice Agent Builder

지시 "예약 문의를 받고 빈 시간에 예약해 줘."

통화 음성 수신 → 말의 내용과 의도 파악

문서에서 정책 검색 → 필요한 추가 질문

예약 API·MCP 호출 → 성공 여부 확인

음성으로 답변 → 녹취·도구 사용·결과 기록

낮은 지연 시간 + 지식 + 도구 + 안전 규칙이 핵심

🧠 개인 인텔리전스

지시 "지난 여행 취향을 반영해 다음 가족 여행을 짜 줘."

허용된 Gmail·Photos·YouTube·Search에서 관련 자료 검색

메일의 일정과 사진의 장소·취향을 함께 연결

현재 검색 정보와 비교해 개인화된 제안 생성

참고한 근거를 보여 주고 사용자가 수정·연결 해제

모델이 나를 새로 학습하는 것보다 필요한 순간 자료를 찾아 쓰는 방식

> **공통 검증.** 음성 에이전트는 잘못 들은 값과 API 실행 결과를, 개인 인텔리전스는 잘못 연결한 기억과 오래된 정보를 사용자가 확인할 수 있어야 합니다.

출처: xAI Voice Agent Builder · Google Gemini Personal Intelligence 공식 발표

FIELD MAP 130개 업종 · 1,300개 공개 표본

업종 이름보다 먼저 볼 것은 업무가 들어와서 끝나는 흐름입니다

🏪 소상공인

100개 업종 · 1,000곳

예약·주문·판매·재고·고객 안내를 중심으로 봤습니다.

2026년 3월 영업 중 상가업소 자료

🏭 제조 중소공장

30개 업종 · 300곳

수주·BOM·자재·생산·검사·출하를 중심으로 봤습니다.

2020년 2월 등록공장 자료

🧭 읽는 방법

상호·업종·주소는 공개자료입니다. 조직과 프로세스는 개별 회사의 사실이 아니라 **업종 표준 운영모델**입니다.

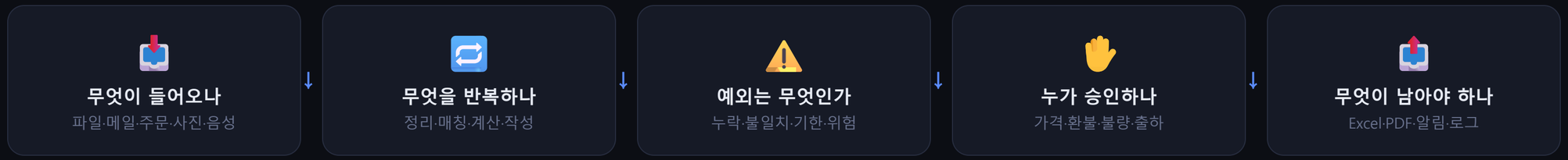
현장 인터뷰 전 자동화 가설

> **주의.** 두 자료는 하나의 확률표본이 아닙니다. 1,300곳을 전국 기업 비율로 일반화하지 않고, 서로 다른 업무 구조와 자동화 시작점을 찾는 데 사용합니다.

출처: 소상공인시장진흥공단 상가(상권)정보 2026-03 · 한국산업단지공단 전국등록공장현황 2020-02

5 QUESTIONS 어떤 업종도 같은 질문으로 시작

자동화 기획은 다섯 칸을 채우면 구체적으로 바뀝니다



🙋 매일 [입력 자료]를 받아 [반복 작업]을 합니다. [예외 기준]은 사람이 확인하고, 최종적으로 [결과물]과 [작업 로그]가 남아야 합니다.

📛 먼저 샘플 파일, 승인 기준, 실패했을 때 멈춰야 하는 지점을 확인하겠습니다.

STARTING POINT 소상공인과 제조업은 시작점이 다르다

같은 AI라도 먼저 줄여야 할 대기와 오류가 다릅니다

소상공인: 오늘의 운영

- 예약·주문·고객 문의를 한곳에 모으기
- 판매와 재고를 비교해 준비·발주 목록 만들기
- 누락·환불·특이 고객만 사람에게 전달하기

첫 실습: 하루 주문 + 재고 → 예외 목록

제조업: 납기와 품질

- 수주서·도면·BOM의 품목과 버전 맞추기
- 재고와 생산계획을 비교해 부족 자재 계산하기
- 검사 결과·불량·출하 승인 근거 남기기

첫 실습: 발주서 + BOM + 재고 → 생산 가능 수량

> **공통점.** AI가 최종 책임자를 대신하는 것이 아니라, 확인해야 할 예외를 줄이고 판단 근거를 한 화면에 모읍니다.

ONE UI 같은 그릇, 다른 파이프라인

기본 UI는 그대로 두고 업종별 처리 규칙만 교체합니다

🍽️ 음식점
판매·예약·재고 업로드 → 메뉴/재료 매칭 → 품절·준비량 확인 → 발주 Excel

🩺 병·의원
예약·문진 입력 → 필수항목 검증 → 누락·주의사항 확인 → 안내문·일정표

🧰 현장 서비스
접수 내용 → 지역·기술·일정 매칭 → 충돌·준비물 확인 → 배정표·체크리스트

🏭 제조업
수주·BOM·재고 업로드 → 품목·버전 매칭 → 부족·납기 검증 → 작업지시·검사 기록

업로드 ↓ **표준화·매칭** ↓ **미리보기·검증** ↓ **사람 승인** ↓ **다운로드·로그**

MEASURE 자동화는 결과값으로 검증

“편해졌다”에서 끝내지 말고 도입 전후 숫자를 남깁니다

⏱️ 시간
처리시간 · 대기시간 · 보고서 작성시간 · 작업당 클릭 수

🎯 정확도
매칭 정확도 · 누락률 · 재작업률 · 검토 후 수정 비율

✋ 사람의 일
전체 검토 건수 · 예외 건수 · 승인 소요시간 · 개입 이유

🏪 운영
예약 누락 · 품절 · 응답시간 · 재고 차이 · 일마감 시간

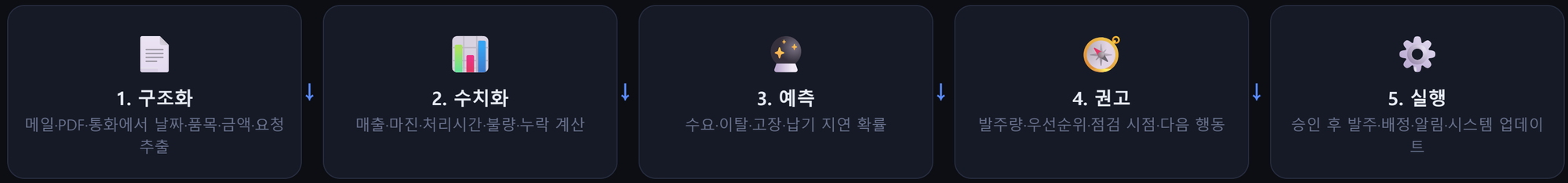
🏭 제조
자재 부족 · 납기 준수 · 불량률 · 계획 변경 · 검사 작성시간

📝 신뢰
판단 근거 기록률 · 오류 복구시간 · 승인 이력 · 재현 가능성

> **논문과 현장의 공통 질문.** 어떤 업무에서, 어떤 조건으로, 누구의 개입을 얼마나 줄였고, 품질과 위험은 어떻게 달라졌는가?

DATA VALUE LADDER 기업은 데이터를 어디까지 바꾸나

같은 데이터도 정리에서 예측과 실행까지 다섯 단계로 발전합니다



LLM이 특히 잘하는 부분

형식이 제각각인 문서·대화에서 항목을 뽑고, 근거와 함께 사람이 읽을 수 있는 설명으로 바꿉니다.

기존 분석이 필요한 부분

정확한 합계·비율은 SQL과 수식, 수요·고장 예측은 통계·머신러닝, 실행은 ERP·CRM·API가 담당합니다.

> **핵심.** LLM 하나가 모든 수치를 만드는 것이 아닙니다. LLM은 비정형 자료와 질문을 연결하고, 계산 엔진·예측 모델·업무 시스템을 오케스트레이션합니다.

REFERENCE CASES 공개 자료에 나온 기업 참고 사례

기업들은 이미 원천 데이터에서 운영 결과까지 연결해 측정하고 있습니다

기업·업무	사용 데이터	만든 수치·판단	공개 결과
Tchibo · 재고	판매·캠페인·품목·채널	품목별 84일 수요 예측	하루 600만 건 예측
Augury · 제조	기계 진동·소리·운전 신호	설비 건강도·고장 원인·조치	기계 고장 75% 감소
Sift Lab · 마케팅	구매·행동·위치·동의 데이터	재구매 확률·세그먼트·CLV	ROAS 40~50% 개선 사례
Gradient Labs · 상담	대화·고객 맥락·정책·도구 결과	해결 여부·절차 준수·만족도	초기 해결률 50%+, CSAT 최대 98%
Block · 사내 데이터	DB·내부 도구·업무 요청	자연어→SQL·분석·업무 실행	엔지니어 75%가 주 8~10시간 절감

> **2026-09-09 참고 사례.** 공급사 고객사례가 공개한 수치이며 독립 실험 결과는 아닙니다. 우리 회사 적용 전에는 같은 지표의 기준선과 계산식을 먼저 고정해야 합니다.

출처: Google Cloud Tchibo·Augury·Sift Lab 사례 · OpenAI Gradient Labs · Anthropic Block 사례

CASE · CUSTOMER VOICE 읽히지 않던 대화가 제품 전략으로

상담 기록은 보관용 파일이 아니라 이탈과 제품 문제를 먼저 알리는 데이터가 됩니다

Kapiche · 수백만 건의 고객 대화

기존 통화·메일·채팅·리뷰 중 일부만 표본 조사
변환 주제·감정·제품 문제·해지 신호·상담 품질
예측 이탈 위험·불만 확산·에스컬레이션 가능성

최근 4주 대화에서 해지 고객이 공통으로 겪은 문제와 첫 징후를 찾아줘.

배송 지연 뒤 두 번째 문의가 발생한 고객군의 이탈 위험이 높습니다. 지역·제품별로 나누겠습니다.

Sourcegraph · 커뮤니티 피드백

기존 게시판·이슈·커뮤니티 의견이 담당자 기억에 의존
변환 반복 요청·불편 단계·경쟁사 언급·감정 변화
기획 제품 우선순위·답변 메시지·경쟁 포지셔닝

요청 빈도보다 업무 중단 영향이 큰 문제를 찾아 다음 분기 개선안으로 묶어줘.

빈도는 낮지만 설치 실패가 사용 시작을 막습니다. 원인 3개와 개선 순서를 제안합니다.

> **교육용 대화 재구성.** 실제 기업의 비공개 프롬프트가 아니라, 공개된 데이터 흐름을 실습에서 재현하도록 만든 질문입니다.

사례 근거: Google Cloud Kapiche AI · Anthropic Sourcegraph 고객사례

FIELD KNOWLEDGE 종이 장부와 정비 경험을 다시 사용

사람의 기억과 오래된 기록을 연결하면 현장에서 바로 묻는 지식 시스템이 됩니다

Sharp 농장 · 1971년부터 쓴 손글씨 장부

기존 과거 파종일·수확량·작업 방식이 종이와 가족 기억에 존재
변환 작물·밭·날짜·수확량·문제·대응을 검색 가능한 기록으로 구성
판단 현재 밭과 비슷한 해의 작업 방식 비교

비가 늦었던 해 중 이 밭과 같은 품종의 파종일과 수확 결과를 비교해줘.

조건이 비슷한 3개 연도를 찾았습니다. 파종 시기·수확량·병해 기록을 함께 보여드릴게요.

KONE · 정비 보고서와 IoT 기록

기존 매뉴얼·과거 수리기록·숙련자 경험이 흩어져 현장 검색이 어려움
변환 증상·모델·오류코드·과거 조치·센서 상태 연결
권고 가능한 원인·확인 순서·필요 부품·상담 이관 기준

이 모델에서 진동과 문 닫힘 오류가 함께 발생한 과거 사례와 첫 점검 순서를 알려줘.

유사 사례 12건 중 8건은 정렬 문제였습니다. 안전 확인 뒤 세 단계로 점검하세요.

> **가능해진 것.** '정답 자동 판정'보다 과거 근거를 빠르게 찾아 초보자도 숙련자의 점검 순서를 따르게 하는 데 먼저 가치가 생깁니다.

사례 근거: OpenAI Small Business Stories — Sharp & Sharp · AWS KONE 사례(4만 명 기술자)

CASE · OPERATIONAL TRACES 종이 기록과 로그가 예방 신호로

문제가 난 뒤 찾던 기록을 계속 분석하면 고장과 사고를 먼저 막는 지표가 됩니다

Breedon · 철도 화차 종이·Excel 기록

기존 이동·잔류 시멘트·정비기록이 누락되거나 뒤늦게 취합
지표 화차별 잔류 발생률·반복 구간·정비 전후 성능
결정 문제가 커지기 전 점검 대상과 우선순위 선정

최근 8주 잔류 발생이 반복된 화차와 정비 후 개선되지 않은 화차를 구분해줘.

반복 위험 7대를 찾았습니다. 빈도·노선·마지막 정비일 기준으로 정렬했습니다.

공개 결과: 데이터 포착 100% · 대상 화차 성능 95% 개선

CMS · 장애 때만 보던 시스템 로그

기존 하루 14~15TB 로그를 저장하지만 사건 발생 뒤 수작업으로 조사
지표 이상 패턴·영향 범위·발생 순서·재발 조건
실행 실시간 경보·관련 로그 재수집·사고 타임라인 생성

정상 기준과 다른 호출 급증이 시작된 시점, 영향을 받은 서비스, 직전 변경을 연결해줘.

14:32부터 인증 오류가 증가했습니다. 직전 배포와 영향 서비스 4개를 표시합니다.

공개 결과: 저장비용 67% 절감 · 2주 수작업 조사 대체

사례 근거: AWS Breedon·CMS 고객사례 · 대화는 교육용 재구성

CONVERSATION DESIGN 요약이 아니라 새로운 신호를 찾는 질문

“정리해 줘”에서 멈추지 말고 비교·선행 신호·예외·행동을 물어봅니다

낮은 가치 질문

고객 상담 내용을 요약해줘.

배송, 가격, 환불 문의가 있었습니다.

무슨 일이 있었는지는 알지만, 다음 행동과 측정 기준이 없습니다.

분석·기획으로 이어지는 질문

재구매 고객과 이탈 고객의 대화를 비교해, 이탈 30일 전에 먼저 나타나는 표현·사건·접촉 횟수를 찾아줘. 근거 대화와 표본 수도 보여줘.

두 번째 배송 문의와 “다시 연락” 표현이 먼저 증가합니다. 검증용 표본과 대응 실험안을 만들겠습니다.

대상 비교
성공군 ↔ 실패군

↓

선행 신호
결과 전에 나타난 것

↓

근거
원문·건수·기간

↓

검증
오탐·누락·재현

↓

실험
누구에게 무엇을 할지

> **복사해 쓸 구조.** “[좋은 결과 집단]과 [나쁜 결과 집단]을 비교해 결과 전에 나타나는 신호를 찾고, 근거·표본 수·오탐 가능성·실행할 실험을 제안해 줘.”

WHAT YOU CAN MEASURE 일반 기업도 뽑을 수 있는 데이터

새로운 데이터를 사기 전에 이미 가진 업무 기록부터 연결합니다

💰 영업·마케팅
입력 견적·주문·CRM·광고·상담
수치 전환율·객단가·마진·재구매·이탈 위험·고객가치

📦 구매·재고
입력 발주·입고·판매·재고·납기
수치 회전율·품절률·과잉재고·공급사 지연·추천 발주량

🏭 생산·품질
입력 작업지시·센서·검사·불량·정지
수치 수율·불량률·OEE·고장 위험·납기 가능성

☎️ 고객서비스
입력 통화·채팅·메일·티켓·환불
수치 응답·해결시간·재접촉·감정·원인·자동해결률

📒 재무·관리
입력 매출·비용·세금계산서·예산
수치 손익·현금흐름·예산차이·미수 위험·이상거래

👥 인사·업무
입력 근태·업무로그·교육·설문
수치 처리량·재작업·대기시간·교육 이수·업무부하

> **개인정보 원칙.** 직원 생산성을 개인 감시 점수로 만들지 않습니다. 목적·접근권한·보존기간을 정하고 가능한 경우 팀·프로세스 단위로 집계합니다.

METRIC DESIGN 숫자는 계산식을 고정해야 비교 가능

AI에게 “분석해 줘”라고 하기 전에 지표의 분자·분모·시간 단위를 정합니다

전환율

구매 완료 고객 ÷ 유효 상담 고객 × 100

중복 고객, 취소, 내부 테스트를 제외할지 정합니다.

재고 회전일

평균 재고 ÷ 기간 매출원가 × 기간 일수

수량 기준인지 원가 기준인지 통일합니다.

정시 납품률

약속일 이내 완료 건 ÷ 전체 납품 건 × 100

고객 변경과 분할 납품 처리 규칙을 정합니다.

상담 자동해결률

사람 이관 없이 해결된 건 ÷ 자동화 대상 건 × 100

재접촉·환불·오답이면 해결에서 제외합니다.

기준선
도입 전 4~8주
↓
파일럿
같은 조건 비교
↓
품질
정확도·오류·예외
↓
성과
시간·비용·매출·위험

> **중요.** 처리시간만 줄고 오류·민원·재작업이 늘면 성공이 아닙니다. 속도 지표와 품질·위험 지표를 한 쌍으로 봅니다.

@MOYI · ASTRA / 2026.09

DATA READINESS 큰 데이터가 없어도 시작하는 순서

처음부터 예측하지 말고 기록의 품질에 맞는 단계부터 시작합니다

1단계 · 문서 몇 건

필드 추출, 분류, 요약, 누락 검증

정답 샘플 20~50건부터

2단계 · 1~3개월

업무량, 처리시간, 오류 원인, 병목 비교

주·월 추세와 예외

3단계 · 12개월+

계절성과 반복 패턴을 반영한 수요·인력·재고 예측

월별 편향과 누락 검증

4단계 · 실시간

센서·주문·이메일 이벤트에 따라 경보와 업무 실행

승인·중단·롤백 필수

🙋 우리 회사에는 엑셀과 메일밖에 없습니다. 어디까지 할 수 있나요?

🤖 먼저 최근 30건에서 날짜·거래처·품목·금액·상태를 추출하고 정확도를 측정하겠습니다. 데이터가 쌓이면 병목과 예측으로 확장하겠습니다.

> **실무 권장치입니다.** 기간과 표본 수는 업무 변동성에 따라 달라집니다. 예측을 시작하기 전 결측·중복·코드 체계·정답 데이터부터 확인합니다.

CAPABILITY MAP 새 기능은 어떤 능력을 키웠나

제품 기능이 늘어난 이유는 모델의 이해력과 실행 장치가 함께 좋아졌기 때문입니다

📚 긴 문맥

많은 자료를 한 번에 참고

긴 보고서·여러 파일·큰 코드 저장소를 이어서 봅니다.

→ Notebook · 프로젝트 · 코드 분석

👁️ 멀티모달

글뿐 아니라 화면·소리도 이해

이미지·표·음성·영상을 함께 읽습니다.

→ OCR · 시각화 · 영상·음성 기능

🧭 추론·계획

복잡한 일을 단계로 나눔

순서를 정하고 중간 결과를 확인합니다.

→ Deep Research · Plan mode

🧰 도구 사용

필요한 도구를 골라 호출

검색·파일·API·코드 실행 결과를 다시 판단합니다.

→ Agent Tools · MCP · 함수 호출

🖱️ 컴퓨터 사용

화면을 보고 클릭·입력

전용 API가 없는 프로그램도 다룹니다.

→ Chrome · 인앱 브라우저 · Office

🔄 기억·오케스트레이션

긴 작업을 이어서 관리

기억·예약·재시도·여러 에이전트를 묶습니다.

→ 예약 작업 · Claude Tag · 다중 에이전트

> **구분.** 긴 문맥·멀티모달·추론은 모델 자체의 능력에 가깝습니다. 기억·예약·도구 연결·다중 에이전트는 모델을 둘러싼 실행 시스템이 더 큰 역할을 합니다.

WHERE THEY DIFFER 같은 LLM도 결과가 달라지는 이유

실제 성능 차이는 모델 하나가 아니라 다섯 층이 함께 만듭니다

1. 기본 모델
추론·언어·코딩·지시 이해

+

2. 입력 범위
문맥 길이·이미지·음성·영상

+

3. 실행 시스템
계획·도구 선택·재시도·기억

+

4. 연결된 서비스
Google·X·Office·브라우저·사내 DB

+

5. 제품 설계
화면·권한·승인·로그·복구

정확도
같은 자료에서 더 맞는 판단을 하는가

속도·비용
필요한 품질을 얼마나 빠르고 싸게 내는가

최신 정보
웹·X·회사 자료를 얼마나 잘 찾는가

실행 안정성
도구를 정확히 쓰고 실패하면 복구하는가

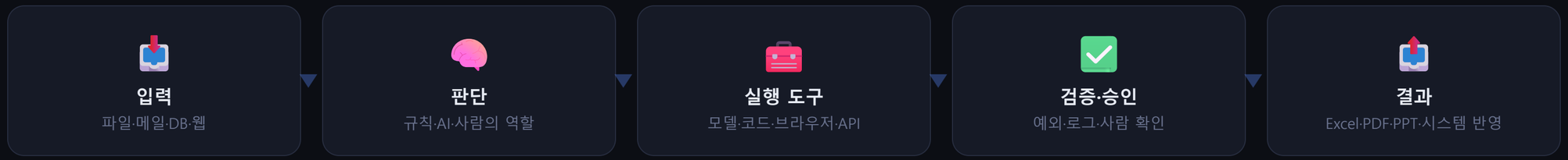
업무 적합성
내 문서·브라우저·시스템 안에서 바로 쓰이는가

> **핵심.** 벤치마크가 높은 모델이 내 업무에서도 항상 가장 좋은 것은 아닙니다. 실제 샘플로 **정확도·속도·비용·실행 성공률·검토 시간**을 함께 비교해야 합니다.

공식 자료: developers.openai.com/api/docs/models · ai.google.dev/gemini-api/docs/long-context · ai.google.dev/gemini-api/docs/function-calling · anthropic.com/news/3-5-models-and-computer-use

WHAT IT MEANS 업데이트가 빨라진 시대의 도입 원칙

도구가 자주 바뀌어도 업무 기준은 남겨야 합니다



1. 모델은 바꿀 수 있게

모델명은 설정으로 분리합니다. 같은 샘플로 품질·비용·속도를 비교한 뒤 바꿉니다.

2. 업무 기준은 그대로

입력 형식, 승인 기준, 결과물과 실패했을 때의 행동은 문서와 테스트로 남깁니다.

3. 월 1회 재검토

새 기능을 쫓아 전부 바꾸지 않고, 현재 병목을 줄이는 업데이트만 작은 실험으로 도입합니다.

> **이 변화가 중요한 이유.** 이제 AI는 답만 하지 않습니다. 자료를 찾고, 결과물을 만들고, 도구를 조작합니다. 따라서 프롬프트만큼 **작업 순서·검증·권한**을 정하는 일이 중요합니다.

대표 공식 자료: openai.com/index/codex-for-almost-everything · anthropic.com/news/create-files · blog.google · firebase.studio · x.ai/news ※ 기능과 제공 범위는 자료 활용 직전 재확인

STILL REQUIRED 강한 모델도 해결하지 못한 부분

모델이 강해져도 대화 기록이 자동으로 업무 기록이 되지는 않습니다

많이 보완된 문제

- 긴 대화에서 요구사항을 놓치는 빈도 감소
- 다단계 계획과 도구 사용의 안정성 향상
- 압축 요약과 작업 재개 품질 향상
- 테스트·검증까지 이어서 수행하는 능력 향상

여전히 남는 문제

- 컨텍스트 크기는 여전히 유한함
- 중요한 정보가 잡음 속에 묻힐 수 있음
- 압축은 세부 내용을 잃을 수 있는 요약임
- 새 세션은 이전 채팅을 자동으로 이어받지 않음

강한 모델 + 명확한 인수인계 + 파일·Git 기록 = 안정적인 장기 작업

> **필수 원칙.** 좋은 모델은 맥락 손실의 확률을 낮춥니다. 문서화는 손실이 발생해도 다시 정확한 상태로 돌아오게 만듭니다.

CONTEXT 현재 대화와 계정 사용량은 다른 것입니다

/status로 현재 작업 상태를 먼저 확인합니다

컨텍스트

지금 답변을 만들 때 펼쳐 둔 대화·파일·지시의 작업 공간입니다. 길어지면 중요한 내용이 묻힐 수 있습니다.

사용량 한도

계정에서 사용할 수 있는 양입니다. 압축하거나 새 대화를 열어도 사용량이 충전되지는 않습니다.

1. 상태 보기
현재 화면의 /status를 확인합니다.

2. 핵심 기록
결정·파일·다음 행동을 먼저 남깁니다.

3. 이어갈 방식
압축할지, 새 작업을 열지 고릅니다.

> **첫날 실습.** 입력창에 /를 쳐서 내 환경에서 제공되는 명령부터 확인합니다. 앱·CLI·버전·권한에 따라 목록이 다릅니다. 공식 명령어 안내 ↗

CONTINUE OR NEW 대화를 정리하는 세 가지 선택

같은 일을 이어갈 때와 새로 시작할 때를 구분합니다

/compact

긴 대화를 핵심 요약으로 접고 같은 업무를 이어갈 때 씁니다. 세부 내용이 빠질 수 있으니 중요한 결정은 파일에도 남깁니다.

새 작업 · CLI /new

다른 업무를 분리하거나 새 관점으로 시작할 때 씁니다. 인계 파일과 결과 파일을 먼저 읽게 합니다.

CLI /clear

새 대화를 시작하면서 터미널 화면도 비우는 CLI 기능입니다. 파일 변경을 되돌리는 명령이 아니며 앱 명령으로 가르치지 않습니다.

새 작업 첫 문장

AGENTS.md와 CURRENT_TASK.md를 읽고, 필요한 결과 파일을 확인해 기록과 실제 상태가 일치하는지 점검한 뒤 다음 행동부터 이어가줘.

SLASH MAP 명령을 외우지 말고 상황에 맞게 씁니다

계획·모델·검토는 필요한 순간에만 꺼냅니다

/plan
범위가 큰 일을 시작하기 전에 순서와 확인할 점을 정합니다.

/model · /reasoning
내 계정에서 가능한 모델과 생각의 강도를 확인합니다. 복잡한 분석에서만 비교합니다.

/fork
현재 대화를 바탕으로 다른 접근을 탐색합니다. 같은 파일을 동시에 고치지 않도록 주의합니다.

/review
수정한 코드의 문제를 찾습니다. 업무 수치나 사실관계 검증을 대신하지는 않습니다.

/status
현재 상태와 사용량 정보를 확인합니다. 컨텍스트와 사용량을 구분합니다.

화면의 /
기능은 환경마다 다릅니다. 목록에 보이는 명령과 설명을 기준으로 실습합니다.

HANDOFF 압축·새 세션 전에 남기는 작업 인수인계

대화가 아니라 다음 작업자가 읽을 수 있는 상태를 남깁니다

🎯 **1. 목표**
최종 결과와 이번 작업 범위

✅ **2. 완료·미완료**
끝난 일과 남은 일을 분리

📌 **3. 결정**
선택한 기준과 바꾸지 않을 것

📁 **4. 파일·Git**
수정 파일, 브랜치, 커밋, 변경 상태

➡️ **5. 다음 행동**
재시작 후 가장 먼저 할 한 단계

```
컨텍스트를 압축하거나 새 세션을 시작하기 전에 인수인계 문서를 작성해줘.
목표 / 완료한 일 / 남은 일 / 확정 결정 / 수정 파일 / 검증 결과 /
실패한 방법 / 다음 한 단계 / 절대 변경하지 않을 것을 포함해줘.
작성 후 현재 Git 상태와 실제 파일을 확인해 빠진 내용이 없는지 검증해줘.
```

> **중요.** AI의 요약만 믿지 말고, 사용자가 결정과 다음 행동을 한 번 확인한 뒤 압축합니다.

DURABLE CONTEXT 새 세션에서도 이어지는 저장소의 기억

중요한 기준은 채팅에 두지 말고 파일과 Git에 저장합니다

AGENTS.md · 작업 규칙
항상 지킬 명령, 금지 범위, 완료와 검증 기준을 저장합니다.

DECISIONS.md · 확정 결정
화면 비율, 파일명, 디자인, 승인 기준처럼 다시 논의하지 않을 결정을 남깁니다.

CURRENT_TASK.md · 현재 인수인계
지금 목표, 진행 상태, 막힌 점, 다음 행동을 짧게 갱신합니다.

Git · 실제 변경 기록
커밋과 diff로 무엇이 바뀌었고 어느 상태가 정상인지 확인합니다.

새 작업을 시작하기 전에 AGENTS.md, DECISIONS.md, CURRENT_TASK.md와 Git 상태를 읽어줘.

바로 수정하지 말고 현재 목표·완료 상태·다음 행동을 먼저 설명해줘.

내 확인을 받은 뒤 남은 작업만 이어서 수행해줘.

> **한 문장 정리.** 컨텍스트는 임시 기억이고, 저장소 문서는 지속되는 기억입니다.

FIRST MD 작업 전에 읽는 기준을 짧게 만듭니다

AGENTS.md에는 계속 지킬 규칙만 둡니다

목표
어떤 결과를 만들지 적습니다.

범위
고칠 파일과 건드리지 않을 영역을 적습니다.

검증
끝내기 전 확인할 검사와 기준을 적습니다.

금지·승인
외부 반영처럼 먼저 확인할 행동을 적습니다.

> **작게 시작합니다.** 모든 규칙을 한 번에 쓰지 않습니다. 한 업무를 끝낸 뒤 다음에도 필요한 기준만 보탭니다.

CURRENT TASK 지금 상태를 다음 작업자에게 넘깁니다

CURRENT_TASK.md에는 오늘의 사실을 짧게 남깁니다

반드시 남길 것

목표 · 확정 결정 · 입력과 결과 파일 · 완료한 일 · 검증 결과 · 미완료 · 다음 행동

남기지 않을 것

비밀번호, 인증 토큰, 개인정보 원문처럼 기록해도 도움이 되지 않는 민감한 내용

```
현재 작업을 다음에도 이어갈 수 있도록 CURRENT_TASK.md에 정리해줘.
목표, 확정 결정, 입력·결과 파일, 완료한 작업, 검증 결과,
미완료와 이유, 다음 행동을 포함하고 실제 파일 상태와 맞는지 확인해줘.
```

SKILL 반복해서 성공한 절차만 재사용합니다

Skill은 업무 이름으로 다시 부르는 절차입니다

성공한 작업
입력·결과·확인 기준이 분명합니다.

작은 범위
회의록 정리처럼 한 업무로 제한합니다.

SKILL.md
name, description, 절차와 검증을 적습니다.

프로젝트 Skill

`.agents/skills/[업무이름]/SKILL.md`에 둡니다. 이름과 description은 언제 쓸지 알아볼 수 있게 씁니다.

처음부터 만들지 않을 일

일회성 조사나 기준이 아직 정해지지 않은 업무는 일반 요청으로 시작해 결과를 먼저 확인합니다.

공식 근거: Codex Skills 만들기

CONFIG 필요한 설정만 확인하고 바꿉니다

처음에는 앱에서 고르고, 반복될 때만 config.toml을 봅니다

처음 쓰는 사람

앱의 모델·추론 선택 메뉴에서 현재 가능한 항목을 확인합니다. 앱과 CLI의 선택이 항상 같다고 가정하지 않습니다.

계속 쓰는 환경

신뢰된 프로젝트에서만 사용자 설정 `~/.codex/config.toml` 또는 프로젝트 `.codex/config.toml`을 검토합니다.

```
# Astra를 선택할 수 있는 환경에서의 최소 예시
model = "gpt-6-astra"
```

> **주의.** 기존 설정 전체를 덮어쓰지 않습니다. 이 한 줄이 모델 권한이나 사용량을 새로 만들지도 않습니다. 공식 설정 안내 ↗

EFFICIENT WORK 빠른 첫 답변보다 다시 하지 않는 작업

효율은 올바른 결과까지 걸린 전체 시간으로 판단합니다

자주 생기는 낭비

매번 모든 문서를 읽기 · 작은 변경마다 전체 검사 · 모든 일에 최대 추론 · 준비 없이 여러 작업으로 나누기

바꾸는 방법

현재 상태와 관련 문서부터 읽기 · 변경 범위의 검증 실행 · 복잡한 판단에서만 강도 비교 · 독립 과제만 나누기

작게 측정합니다

같은 난도의 샘플에서 처리 시간, 반복 설명 횟수, 재작업 횟수, 검증 누락을 함께 기록합니다.

09 AI 에이전트

챗봇에서 에이전트로의 진화 — 단순한 '상담원'과 실제 '일을 맡길 수 있는 직원'의 차이

🪖 챗봇 — 질문에 한 번 대답하고 끝납니다.

🙋 회의록 요약해줘

🤖 (회의록을 요약하여) 핵심 3가지는 ①출시일, ②예산, ③마케팅 방안입니다. (작업 종료)

🤖 (사용자가 '이걸 팀원들에게 메일로도 보내줘'라고 하면) '저는 메일 발송 기능이 없으니 그건 직접 하셔야 합니다.'라고 거절합니다.

→ 이처럼 챗봇은 사용자가 묻는 말에 정보만 제공하고 끝나는 '상담원' 역할에 그칩니다.

🤖 AI 에이전트 — 주어진 목표를 달성할 때까지 스스로 일합니다.

🙋 경쟁사 3곳 조사해 보고서 만들어 메일로 보내줘

🤖 에이전트 스스로 ①검색 → ②내용 정리 → ③보고서 작성 → ④이메일 발송의 순서를 계획하고 하나씩 실행합니다. ✓

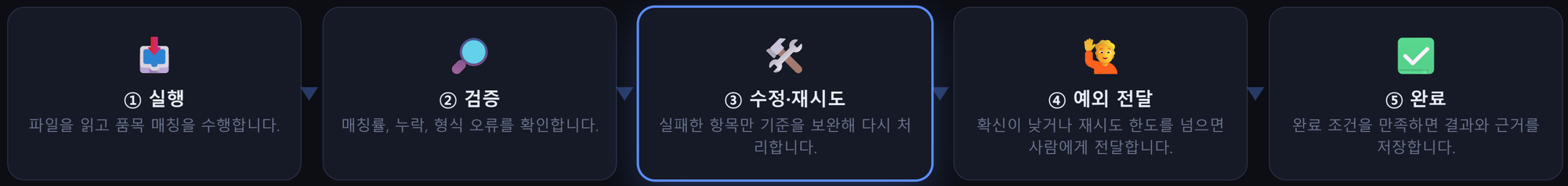
🤖 최종적으로 '메일 발송 준비가 완료되었습니다. 발송 버튼만 눌러주세요.'라며 작업의 끝을 맺습니다.

→ 에이전트는 하나의 업무를 통째로 맡기면 목표 달성 시점까지 완수해 내는 '실무 직원'과 같습니다.

LOOP 한 번 실행이 아니라, 결과를 보고 다음 행동을 바꾸는 반복

루프는 같은 일을 되풀이하는 것이 아니라, 결과를 확인하고 실패 원인을 고쳐 다시 실행하는 구조입니다.

품목 매칭이라면 전체 파일을 무작정 반복하지 않습니다. 검증에서 탈락한 항목만 다시 처리하고, 기준을 넘지 못하면 사람에게 넘깁니다.



🛑 완료 조건

예: 매칭률 95% 이상, 필수 열 누락 0건처럼 멈출 기준을 숫자로 정합니다.

🔢 재시도 한도

같은 오류를 무한 반복하지 않도록 최대 횟수와 시간 제한을 둡니다.

👤 사람 승인

외부 발송, 확정 저장, 결제처럼 위험한 행동 앞에서는 반드시 승인을 받습니다.

> **한 문장 정리:** 워크플로우가 정해진 순서를 실행한다면, 루프는 실행 결과에 따라 다음 행동을 바꾸면서 완료 조건에 도달하도록 만듭니다.

데이터 전달·승인 지점을 지휘하는 것입니다.

한 명의 총괄 에이전트가 모든 일을 직접 하는 대신, 전문 작업자와 도구에 일을 나누고 결과를 모아 하나의 업무를 완성합니다.



총괄 오케스트레이터

업무를 분해하고 담당을 배정하며, 실패 시 재시도·대체 경로·사람 승인을 결정합니다.

📥 수집 담당

메일과 폴더에서 파일을 모으고 작업 ID를 발급합니다.

🧹 정리 담당

열 이름, 날짜, 단위와 품목명을 표준화합니다.

🔗 매칭 담당

기준 자료와 품목을 연결하고 신뢰도를 계산합니다.

🛡️ 검증 담당

누락과 이상치를 찾아 재시도 또는 사람 검토로 보냅니다.

📊 보고 담당

Excel·PDF 결과와 팀장용 요약을 만듭니다.

📝 기록 담당

판단 근거, 수정 내역, 승인자를 로그로 남깁니다.

🔁 루프

한 작업 안에서 결과를 확인하고 수정·재시도합니다. 예: 미매칭 17건만 다시 처리.

🎼 오케스트레이션

여러 작업자 사이에서 담당과 순서를 조정합니다. 예: 수집→매칭→검증→보고→승인.

도입 판단 워크플로우·루프·오케스트레이션 중 무엇이 필요한가

처음부터 여러 에이전트를 만들지 말고, 업무의 불확실성만큼만 자율성을 줍니다

순서가 항상 같다

입력과 규칙이 명확하고 같은 단계로 처리됩니다.

선택: 워크플로우

예: 파일명 변경 → 폴더 이동 → 메일 첨부

결과를 보고 다시 판단한다

검증 결과에 따라 수정하거나 일부 항목을 재처리합니다.

선택: 에이전트 루프

예: 미매칭 품목만 기준을 바꿔 재시도

역할과 도구가 여러 개다

수집·판단·검증·보고 담당의 순서와 전달을 조정합니다.

선택: 오케스트레이션

예: 주문 수집 → 재고 비교 → 승인 → 보고

> **용어 정리.** 오케스트라는 여러 연주자가 함께 연주하는 조직을 뜻하는 비유입니다. 시스템에서 역할과 실행 순서를 조율하는 구조의 정확한 명칭은 **오케스트레이션(Orchestration)**입니다.

필수 기능 실제로 작동하는 오케스트레이션의 구성

에이전트 수보다 먼저 만들어야 할 것은 일의 전달·검증·중단 기능입니다

1. 작업 접수
파일·요청을 받고 작업 ID와 담당 흐름을 생성합니다.

2. 상태와 순서
대기·처리 중·검토·완료·실패 상태와 다음 단계를 관리합니다.

3. 데이터 전달
앞 단계의 출력 형식을 고정해 다음 역할에 정확히 넘깁니다.

4. 검증과 재시도
완료 조건을 검사하고 실패한 항목만 제한적으로 다시 실행합니다.

5. 사람 승인
외부 발송·확정 저장·결제 전에 미리보기와 승인 버튼을 둡니다.

6. 로그와 복구
입력·판단 근거·수정·승인을 기록하고 이전 상태로 되돌립니다.

```
작업 ID: JOB-2026-0717-03  상태: 검토 필요
수집 ✓  표준화 ✓  매칭 △  검증 대기  보고 중지
예외: 신뢰도 70% 미만 17건 → 담당자 승인 요청
```

> **좋은 시스템의 모습.** 모든 일을 조용히 자동 처리하는 것이 아니라, 어디까지 처리했고 왜 멈췄는지 사람이 바로 이해할 수 있어야 합니다.

도입 순서 작은 자동화에서 실제 운영까지

오케스트레이션은 한 번에 구축하지 않고 고정 흐름에서 예외 처리까지 단계적으로 확장합니다



1. 업무 분해

입력·규칙·판단·결과·승인 지점을 적습니다.



2. 고정 흐름

AI 없이도 설명 가능한 순서를 먼저 자동화합니다.



3. 판단 루프

불확실한 한 단계에만 AI 판단과 재시도를 넣습니다.



4. 역할 분리

업무량과 전문성이 필요할 때 담당 역할을 나눕니다.



5. 제한 운영

샘플·내부 사용자·사람 승인으로 시작해 범위를 늘립니다.

시작 조건

반복량이 많고 입력·출력을 설명할 수 있다.

확대 조건

성공률·검토 건수·처리 시간을 로그로 확인했다.

중단 조건

같은 오류 반복, 비용 초과, 보안·승인 기준 위반.

> **첫 도입 권장 범위.** 하루 업무 전체가 아니라 "주문 파일과 재고 파일을 비교해 검토 목록을 만드는 단계" 하나부터 시작합니다.

09 AI 처리 과정

사용자의 지시가 들어왔을 때, AI 내부에서는 다음과 같은 처리 과정이 일어납니다.

📋 지시: ▌

📥 입력 0 💭 사고 0 📤 출력 0 합계 0 ≈ 0원

⏳ 📥 **입력 이해** 입력된 프롬프트에서 사용자의 진짜 의도, 관련 참석자, 핵심 주제를 파악합니다. +사고 187

⏳ 🧭 **처리 계획** 요청을 해결하기 위해 어떤 순서로 작업할지(예: 6단계 작업) 스스로 계획을 수립합니다. +사고 312

⏳ 🔎 **텍스트 분석** 텍스트 데이터를 분석하여 누가, 언제, 어떤 액션 아이템을 담당하기로 했는지 추출해 냅니다. +사고 1,458

⏳ ✅ **자기 검증** 작성된 초안에서 누락된 정보나 형식의 오류, 마감기한 명시 여부 등을 스스로 확인하고 보완합니다. +사고 1,420

⏳ 💬 **답변 생성** 최종 확정된 내용을 바탕으로 한 단어씩 차례대로 텍스트를 생성하여 답변을 완성합니다. +사고 443·출력 142

실전 Real Case · 실제 작업

실제 개발 환경에서 AI 에이전트에게 업무를 지시하는 생생한 예시입니다. (웹 화면의 데이터 표 기능 수정 요청)

Claude Code — nenova-erp / revenue-management.js

나

'현재 영업매출관리 화면의 데이터 표가 좌우로도 길고 상하로도 길어서 보기 힘든데, 가로 스크롤바가 표 맨 아래쪽에만 있어서 불편해.' **상단에도** 추가해 주고, 화면을 상하좌우로 스크롤하더라도 **업체통용명 · 차수가 고정**열은 화면에 고정되어 항상 보이게 만들어 줘.'

Claude

- 🔎 **검색** 가장 먼저 프로젝트 파일들을 검색하여 'revenue-management.js' 파일 안에서 해당 표가 그려지는 코드 위치(518번째 줄 주변)를 찾아냅니다. ✓
- 📖 **읽기** 해당 부분의 HTML 구조와 적용된 CSS 스타일 클래스들을 읽어보며 현재 어떻게 구현되어 있는지 파악합니다. ✓
- 🖍️ **수정** 파악한 코드를 바탕으로 표 상단에 가로 스크롤 영역을 새로 만들고, 상단 스크롤을 움직이면 하단 스크롤도 같이 움직이도록 자바스크립트 이벤트를 연결합니다. ✓
- 🖍️ **수정** 업체통용명 열 `sticky-left` · 차수 헤더 `sticky-top` ✓
- ✅ **검증** 코드 수정 후 오류가 발생하지 않는지 프로젝트 빌드를 돌려 자체 검증을 마친 후 → 📦 **커밋·배포** ✓

✓ 완료 — 표 위 가로 스크롤바(하단과 동기화) + **업체통용명 열 고정** + **차수 헤더 고정**처리해 두었습니다. 이제 화면을 상하좌우 어디로 스크롤해도 기준 정보가 따라오며 보입니다. 수정된 코드는 Git에 커밋(1651f41) 후 배포까지 마쳤으니, 브라우저를 새로고침하여 확인해 보세요. 혹시 고정된 열의 너비나 표의 전체 높이를 조정하고 싶으시면 편하게 말씀해 주세요.'

💡 **이 예시의 핵심:** *사용자가 겪는 불편한 현상과 원하는 결과를 구체적으로 설명하기만 하면(어디가 어떻게 불편하고, 무엇을 고정하고 싶은지), AI 에이전트가 스스로 원인 코드를 찾고, 수정하고, 테스트한 뒤, 실제 서버 배포까지 원스톱으로 처리해 줍니다.* **기존에는 기획자, 디자이너, 개발자가 모여 처리해야 했던 업무가 이제는 AI와의 대화 한 번으로 끝날 수 있습니다.**

현실 광고 속 에이전트 vs 실제 에이전트

우리가 흔히 상상하는 'AI가 다 알아서 해주는' 마법 같은 결과 뒤에는 치밀하게 쪼개고 설계된 여러 개의 업무 단계가 숨어 있습니다.

📺 광고에서는



사용자

신규 고객을 분석하고 제안서를 만들어 이메일까지 보내줘.



AI 에이전트

완료했습니다. 고객 분석, 제안서 작성, 이메일 발송까지 마쳤습니다.

사용자의 말 한마디에 AI가 마법처럼 모든 것을 한 번에 뚝딱 끝내는 것처럼 보입니다.

⚙️ 실제 내부에서는

① 신규 고객 목록·CRM 조회

② 회사 정보 검색·영업 가능성 평가

③ 기존 제안서·가격 정책 확인

④ 제안서와 이메일 초안 생성

⑤ 담당자 승인 요청

⑥ 승인 후 발송·CRM 기록

실제로는 명확한 작업 단계, 연결된 도구, 비즈니스 규칙, 그리고 사람의 승인 과정이 톱니바퀴처럼 맞물려 돌아가야 합니다.

💡 **잊지 말아야 할 핵심:** *실무에서 쓰이는 AI 에이전트는 척하면 척 알아듣는 만능 천재 직원이 아닙니다. 명확히 주어진 절차에 따라, 쥐어준 도구를 활용해 묵묵히 일을 수행하고 그 결과를 보고하는 '디지털 실무자'로 설계해야만 실패하지 않습니다.*

구조 AI를 직원처럼 만드는 5가지

단순히 프롬프트에 '너는 지금부터 유능한 마케터야'라고 역할만 부여한다고 해서 진짜 실무자가 되지는 않습니다.

역할만 정해서는 부족합니다. 데이터·도구·규칙을 함께 연결해야 실제로 일을 처리하는 에이전트가 됩니다.



역할

어떤 일을 해야 하고 어디까지 권한이 있는지, 절대 해서는 안 될 일은 무엇인지 명확히 규정

+



회사 자료

업무 매뉴얼, 제품 마스터 데이터, 과거의 성공적인 결과물 등 작업에 필요한 지식 데이터 제공

+



업무 도구

엑셀 파일 제어, CRM/ERP 시스템 조회, 이메일 발송 등 실제 액션을 취할 수 있는 소프트웨어 도구 연동

+



실행 규칙

업무를 처리하는 순서, 언제 이 일을 끝마쳤다고 볼 수 있는지(완료 조건), 예외 상황 발생 시 대처법

+



검증·승인

작업 기록(로그) 남기기, 중요 단계에서 사람의 결재 받기, 문제 발생 시 이전 상태로 되돌리기(롤백)

직무기술서

시스템 프롬프트

교육 자료

사내 문서·RAG

업무 프로그램

API·도구 연결

결재선

사람 승인 단계

업무 일지

작업 로그·이력

핵심 구성: *역할을 정하고 회사 데이터와 업무 도구를 연결합니다. 여기에 실행 규칙과 검증·승인 절차를 더해야 실무에서 쓸 수 있습니다.*

실행 주간 판매 보고서 에이전트

AI가 스스로 시스템 도구를 사용하고, 그 결과를 분석하여 다음 단계로 무엇을 할지 스스로 판단하고 행동하는 실제 실행 과정을 살펴봅니다.

🎯 **목표 확인**
이번 주 판매 보고서 완성

↓

🛠️ **도구 실행**
판매 DB·파일 보관함 조회

↓

∞ **결과 관찰**
데이터를 분석해 보니 주요 거래처 2곳의 데이터가 누락된 것을 스스로 인지함.

↓

🧠 **다음 행동**
누락된 자료를 찾기 위해 사용자 이메일의 수신함 첨부파일을 검색해 보기로 결정함.

weekly-report-agent.log

[조회]	판매 데이터베이스 · 이번 주	1,034건 ✓
[관찰]	거래처 A·B 자료 없음	누락 2
[검색]	이메일 첨부파일 검색	2개 발견 ✓
[처리]	표준화·품목 매칭	17건 검토 필요
[대기]	담당자에게 검토 요청	사람 승인

승인 후 계속: 담당자가 승인하면 잠시 대기했던 작업을 재개하여 엑셀/PDF 보고서를 만들고, 수치 합계가 맞는지 자체 검증한 후, 메신저로 팀에 공유하고 작업 로그를 남기며 마무리함.

💡 **에이전트 루프의 핵심 작동 원리:** *최초의 '목표'가 주어지면 스스로 '계획'을 세우고, '도구'를 실행한 뒤 '결과를 관찰'하고 '검증'하여 '다음 행동'을 결정하는 이 사이클을, 부여받은 완료 조건이 충족될 때까지 쉼 없이 반복하는 것이 에이전트의 본질입니다.*

반복 업무는 스스로 척척 처리하되, 판단이 어렵거나 예외적인 상황은 조용히 사람에게 넘겨 확인을 받습니다.

AUTO

AI 자동 실행

판단 기준이 명확하고 속도가 생명인 반복적인 작업

- ✓ 파일 읽기·열 표준화
- ✓ 데이터 분류·품목 추천
- ✓ 보고서·이메일 초안

REVIEW

사람 검토

AI의 확신도가 낮거나, 결론이 비즈니스에 미치는 영향이 큰 작업

- ⚠️ 낮은 신뢰도 매칭
- ⚠️ 예외·경계값 판단
- ⚠️ 외부 발송 전 승인

BLOCK

강한 승인

한 번 실행하면 수습하기 어렵거나 재무적/보안적 위험이 따르는 작업

- 🔒 계약 조건 변경
- 🔒 금액 지급·환불
- 🔒 대량 삭제·권한 변경

👁️ 미리보기

실행 전에 어떤 자료로 무엇을 만들지 먼저 보여 줍니다.

✋ 승인

메일 발송·결제처럼 외부에 영향을 주는 일은 담당자가 확인한 뒤 실행합니다.

📝 로그

↩ 롤백

09 통합 대시보드

에이전트 운영의 4 안전장치

'AI를 믿되, 그 과정은 반드시 사람 눈에 보이게 만들어라.' — AI가 뒤에서 무슨 일을 하는지 투명하게 보이지 않으면, 사고가 났을 때 원인을 파악할 수도 복구할 수도 없습니다.



작업 로그

AI가 언제, 어떤 작업을, 무슨 근거로 수행했는지 텍스트 형태로 상세히 남기는 기록장입니다.

필요한 이유: 오류가 발생했을 때 과거 상황을 재현하고 정확한 원인을 추적하기 위해 필수적입니다.



히스토리

시스템의 데이터나 파일 상태가 어떻게 변해왔는지 시간 순서대로 차곡차곡 쌓아둔 버전 기록입니다.

필요한 이유: AI가 특정 상황에서 반복적으로 하는 실수 패턴을 분석하고 모델을 교정하기 위함입니다.



롤백 지점

데이터를 덮어쓰거나 대량 삭제하는 등 위험한 작업을 수행하기 직전에 시스템이 자동으로 남겨두는 백업 지점입니다.

필요한 이유: 대형 사고가 터지더라도 버튼 한 번으로 사고 이전의 안전한 상태로 즉시 되돌리기 위해서입니다.



시각 대시보드

현재 AI 에이전트들이 어떤 작업을 하고 있고 어디까지 진행되었는지 시각적인 그래프나 수치로 보여주는 현황판입니다.

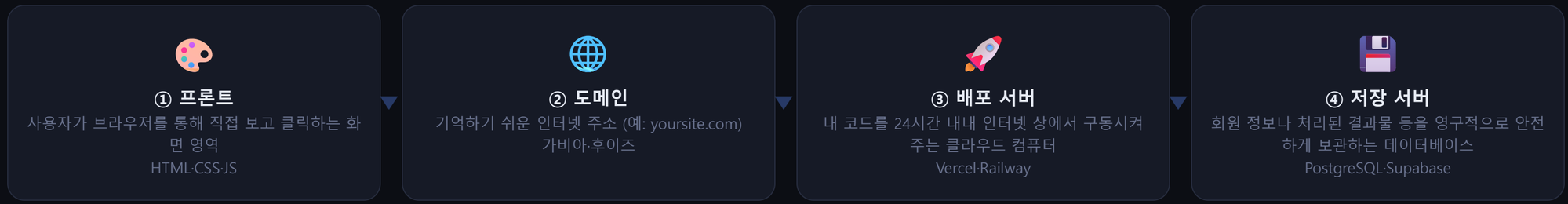
필요한 이유: 관리자가 전체 상황을 직관적으로 파악하고, 이상 징후가 보일 때 즉각적으로 개입하여 작업을 중단시키기 위함입니다.

> 안전한 화면은 세 가지를 바로 알려 줍니다. **AI가 무엇을 했는지, 언제 문제가 시작됐는지, 어디까지 되돌릴 수 있는지**가 보여야 합니다.

09 나만의 사이트

내 PC(로컬)에서만 돌던 자동화 코드를 → 인터넷 접속이 가능한 '나만의 웹 서비스'로 배포하는 과정

다른 팀원이나 외부인도 언제든 이 시스템을 이용하게 만들려면, 눈에 보이는 화면(프론트), 주소(도메인), 프로그램을 돌릴 컴퓨터(배포 서버), 그리고 데이터를 담아둘 창고(저장 서버)가 필요합니다.



> 전체적인 연결 흐름: 사용자가 도메인 주소로 접속 → 배포 서버가 프론트 화면을 보여줌 → 화면에서 API를 호출 → 백엔드가 저장 서버에서 데이터를 꺼냄 → 최종 결과를 사용자 화면에 표시. (참고로 지금 보고 있는 이 학습 사이트 자체도 GitHub Pages라는 배포 서버를 이용한 동일한 구조로 만들어져 있습니다.)

AI 시대를 맞이한 소프트웨어 인터페이스의 패러다임이 UI에서 UX로, 그리고 이제는 AX로 진화하고 있습니다.

사용자가 화면을 일일이 **클릭**하며 조작하던 시대(UI)를 지나, 편리한 **사용 경험**을 정교하게 설계하는 시대(UX)를 거쳐, 이제는 **AI 에이전트가 사람의 의도를 파악해 대신 일해주는** 새로운 차원의 시대(AX)로 넘어가고 있습니다.

PAST

UI

User Interface (사용자 인터페이스) · '화면을 보고 직접 누른다'

- 🖱️ 화면상의 버튼, 메뉴, 입력 폼 등을 사람이 일일이 찾아 클릭하고 채워 넣는 방식
- 👁️ 기능을 어떻게 보기 좋게, 예쁜 화면과 아이콘으로 배치할 것인지가 주된 관심사
- 📋 "'사용자 눈에 어떻게 보여줄 것인가?'에 집중"

NOW

UX

User Experience (사용자 경험) · '막힘없이 쉽고 편하게 쓴다'

- 🧭 단순한 시각적 배치를 넘어, 사용자가 목적을 달성하는 전체 흐름과 편의성, 심리적 만족감까지 고려한 설계
- 💬 복잡한 메뉴 대신 대화형 챗봇이나 자연어로 검색하는 방식이 도입되기 시작함
- 🎯 "'사용자가 시스템을 쓰면서 어떻게 느끼고 얼마나 편안해하는가?'에 집중"

NEXT

AX

Agent Experience (에이전트 경험) · '이제는 AI가 알아서 대신 처리한다'

- 🤖 사용자는 최종 목표만 툭 던져주고, AI 에이전트가 내부적으로 도구를 활용해 스스로 작업을 수행하고 결과만 보고함
- 📜 화려한 화면보다 **AI가 데이터를 읽고 도구를 쓰게 만드는 구조와 명세(스키마·MCP)** 가 더 중요해졌습니다.
- 🤝 소프트웨어를 '내가 직접 다루는 편리한 도구'로 보는 것을 넘어, '나와 협업하는 똑똑한 동료'로 경험하게 만드는 설계

💡 무엇이 바뀌나

사람이 보는 화면은 단순하고 알아보기 쉽게 만듭니다. 반면 뒷단의 데이터와 도구는 AI가 정확히 읽고 조작할 수 있도록 구조화해야 합니다.

📊 왜 지금

리서치 기관의 전망에 따르면, 2026년에는 글로벌 기업용 애플리케이션의 약 **40% 이상이 업무용 AI 에이전트를 기본으로 탑재할 것** 으로 예상됩니다(2025년 기준 5% 미만). 시스템에 얼마나 훌륭한 AX(에이전트 경험)를 녹여내느냐가 곧 기업 솔루션의 새로운 경쟁력

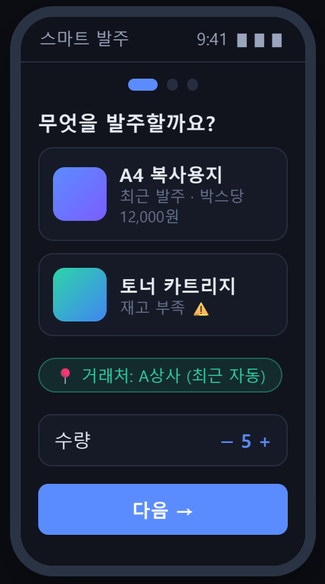
예시 화면 비교 · 모바일

똑같은 '물품 발주 업무'를 모바일 앱에서 처리할 때, UI, UX, AX 방식의 확연한 차이를 비교해 봅니다.



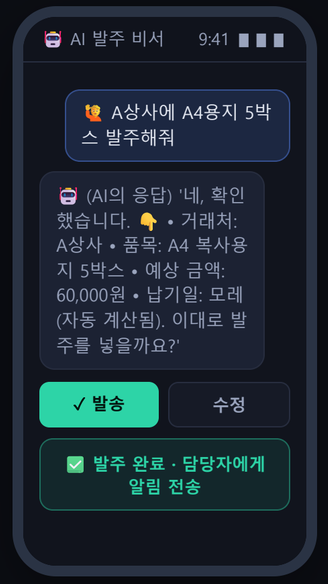
UI

데이터베이스의 필드를 그대로 나열한 형태. 사용자가 직접 7개 이상의 빈칸을 고민하며 일일이 채워 넣어야 합니다.



UX

사용자의 과거 발주 이력을 바탕으로 자주 쓰는 품목, 추천 거래처, 예상 납기일 등이 기본값으로 채워져 있어, 터치 몇 번만으로 빠르고 편하게 주문을 넘길 수 있습니다.



AX

자연어로 대충 말해도 AI 시스템이 빈칸의 의미를 스스로 추론하여 발주서를 완성하고, 사람은 최종 결과만 훑어본 후 승인 버튼만 누르면 모든 작업이 끝납니다.

💡 **사용자 인터페이스 발전 흐름:** *일일이 7칸을 입력하는 수작업(UI) → 똑똑한 추천을 통해 1~2번의 탭으로 단축(UX) → 말 한마디면 AI가 모든 칸을 채우고 사람은 승인만 하는 단계(AX). 소프트웨어가 발전할수록 사용자가 화면에 대고 해야 할 '육체적/인지적 노동'은 극단적으로 줄어듭니다.*

채널 ▾ 선택

항목 ☐ 매출 ☐ 원가 ☐ 이익

정렬 ▾ 선택

형식 ▾ 표 / 차트

생성

UI

조회 기간, 데이터 항목, 정렬 방식 등 보고서 생성을 위한 수많은 옵션을 사용자가 드롭다운 메뉴에서 일일이 찾아 선택해야 합니다.

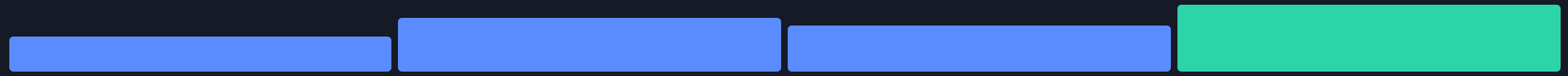
erp.com/insights

3분기 대시보드

자동 추천

매출 +12%

이익 +8%



UX

복잡한 옵션 선택 없이, 접속하자마자 이번 분기의 핵심 매출 지표와 트렌드 차트가 시각적으로 보기 좋게 자동으로 표시됩니다.

erp.com/ai

🙋 3분기 매출 보고서 만들어 팀에 공유해줘

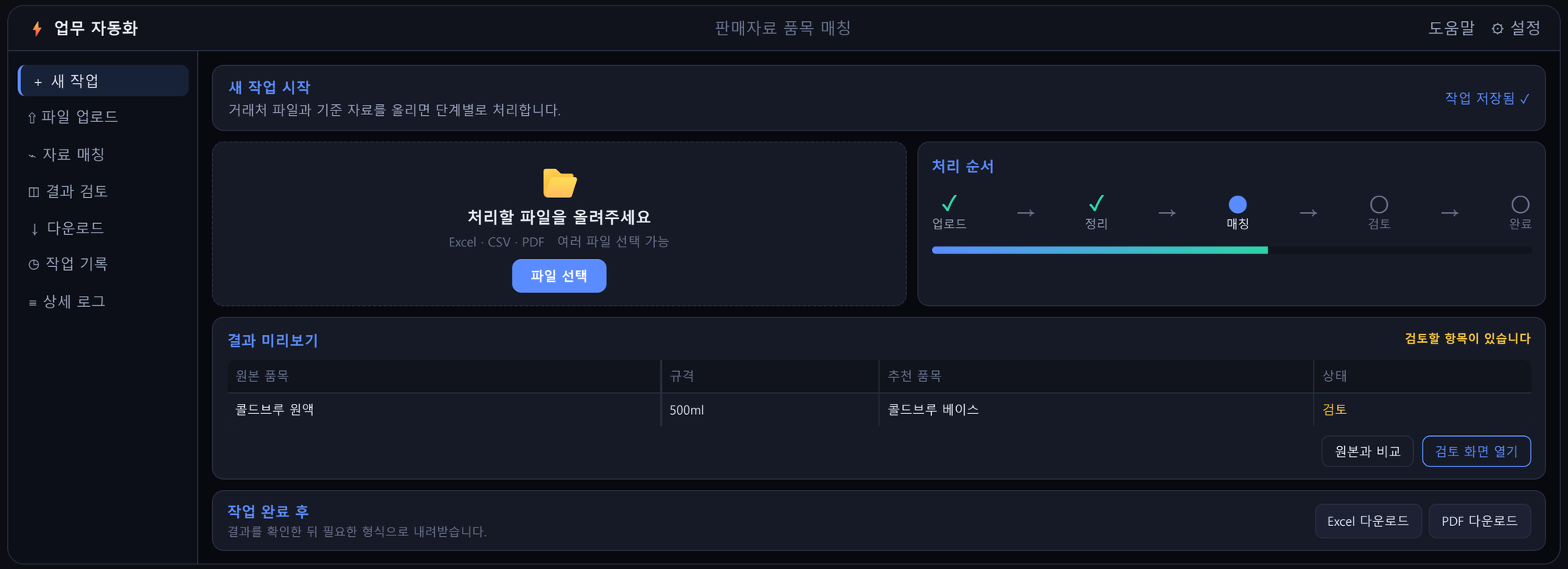
🤖 ① 데이터 집계 ✓ ② 표·차트 생성 ✓ ③ 요약 작성 ✓

🤖 (에이전트의 응답) '요청하신 3분기 매출 분석 보고서가 완성되었습니다. 주요 요약: 전분기 대비 매출 12% 상승, 상위 품목 A와 B가 실적을 견인했습니다.'

실습 업무 자동화 기본 UI

통계보다 먼저, 작업 흐름이 한눈에 보이는 화면

파일을 올리고, 현재 단계를 확인하고, 결과를 검토한 뒤 바로 내려받는 것이 기본입니다.

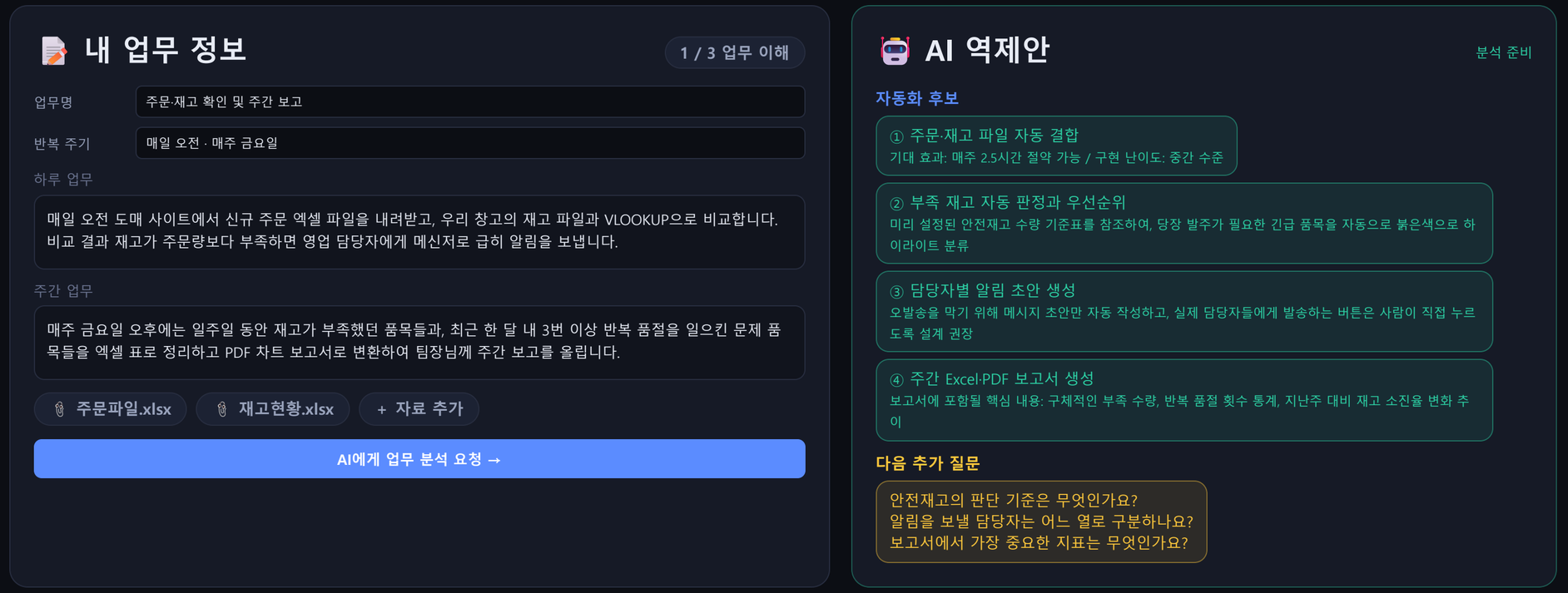


화면의 중심. *숫자 통계가 아니라 지금 무엇을 해야 하는지, 어디까지 처리됐는지, 무엇을 검토하고 내려받을지를 바로 보여줍니다.*

입력 AI 업무 설계

반복 업무를 적으면 AI가 자동화할 부분부터 먼저 제안합니다

개발을 위한 정확한 요구사항 기획서가 없어도 전혀 걱정하지 마세요. 평소 일하는 방식을 줄글로 편하게 적기만 하면, 똑똑한 AI 모델이 어떤 부분을 자동화하면 좋을지 후보를 뽑아주고, 모호한 부분을 구체화하기 위한 추가 질문까지 던져줍니다.



💡 **기획 자동화의 흐름:** *사용자가 평소 하던 업무를 줄글로 입력함 → AI가 구조를 분석해 자동화 후보 기능 역제안 → 모호한 정책을 사용자에게 추가 질문하여 확정 → 최종적으로 깔끔한 시스템 요구사항 명세서로 자동 작성 → 이를 바탕으로 앞서 본 공통 웹 화면 UI에 알맞은 기능을 덧붙여 실제 프로그램을 완성.*

실무에서 대형 사고 없이 AI를 안전하게 다루기 위해 반드시 알아야 할 4가지 보안 및 윤리 기본기

성능을 끝까지 끌어올리는 것보다, 사고 없이 안전하게 쓰는 방법을 먼저 익혀야 합니다.

😵 환각

🙋 ○○법 몇 조에 있어?

🤖 제12조에 있습니다

⚠️ 주의: AI는 그럴듯한 거짓말(환각)을 할 수 있으므로, 실제로는 존재하지 않는 가짜 조항을 지어냈을 확률이 있습니다.

→ 해결책: AI에게 항상 답변의 출처 링크를 요구하고, 국가법령정보센터 등 신뢰할 수 있는 1차 자료를 통해 사람이 직접 크로스체크해야 합니다.

🔒 보안

🙋 [고객 주민번호 명단] 분석해줘

⚠️ 치명적 위험: 주민번호, 전화번호, 회사 미공개 재무 정보 등을 일반 퍼블릭 AI 챗봇에 입력하면 그 데이터가 외부 서버에 학습용으로 남거나 유출될 수 있습니다.

→ 해결책: 민감한 정보는 엑셀에서 미리 가짜 데이터로 변환(마스킹/가명화)한 뒤 분석을 맡기거나, 학습에 데이터가 활용되지 않는 비공개 기업용 엔터프라이즈 플랜을 사용해야 합니다.

🗓️ 한계·컷오프

🙋 어제 발표된 그 뉴스 알려줘

🤖 제 데이터 학습은 작년 말에 멈춰 있기 때문에, 어제 일어난 최신 뉴스는 알 수 없습니다.

→ 해결책: 최신 정보가 필요할 때는 웹 검색 기능이 켜져 있는 도구를 사용하거나, 최신 문서를 직접 첨부해서 읽게 하는(RAG) 방식을 써야 합니다.

⚖️ 저작권·윤리

🙋 이 생성 이미지 상업적으로 써도 돼?

🤖 AI가 생성한 결과물은 기존 저작물을 학습하여 조합한 것이므로 저작권 귀속이 모호하며, 타인의 저작권을 침해했을 가능성이 존재합니다.

→ 해결책: AI 생성물을 그대로 복사해서 상업적 광고 등에 쓰지 말고, 사람이 아이디어 수준으로만 참고하여 새로 편집하거나 사내 저작권 가이드라인을 엄격히 준수해야 합니다.

> 결론 요약: ① AI의 말은 항상 의심하고 팩트 체크할 것, ② 회사의 기밀 데이터는 절대 통째로 넘기지 말 것, ③ AI 모델의 지식 한계를 명확히 알고 외부 도구로 영리하게 보완할 것, ④ 생성물의 저작권과 회사 규정을 반드시 지킬 것.

심화 스키마 (Schema)

스키마(Schema)의 개념 — 데이터가 반드시 따라야 하는 깐깐한 '구조적 규칙서' 혹은 '설계도'

스키마는 받을 **데이터 항목**, 각 항목의 **형식**, **필수 여부**를 미리 정한 표입니다. AI의 출력을 이 형식에 맞추면 누락과 형식 오류를 처음부터 줄일 수 있습니다.

Fig 1. Schema 정의

```
contact_schema = { name : string (필수) // 이름 phone : string (필수) // 010-####-#### amount: number (필수) // ≥ 0 date : string (선택) // YYYY-MM-DD }
```

Fig 2. 인스턴스 검증 (validate)

```
{ name:"김철수", phone:"010-1234-5678", amount:12000 }   ✓ 적합
```

```
{ name:"김철수", phone:"전화없음", amount:"공짜" }   ✗ 위반
└ 실패 사유: phone 필드에 전화번호 형식이 아닌 텍스트가 들어갔고, amount 필드에 숫자 타입이 아닌 문자열('공짜')이 들어가 스키마 규칙을 2개 위반했습니다.
```

Fig 3. AI에서 스키마의 3가지 역할

📐 출력 강제

"'반드시 이 JSON 스키마 구조로만 대답해'라고 지시하면 → AI가 정해진 형틀에 정확히 들어맞는 깔끔한 구조화 데이터를 출력합니다."

🛡️ 검증 게이트

AI가 스키마에 어긋난 엉터리 형식으로 답변을 내놓으면, 시스템 게이트가 이를 즉시 거부하고 AI에게 에러 메시지를 던져 재작업을 강제합니다. 즉, 불량품은 절대 다음 단계로 넘어가지 못합니다.

🏗️ 구조 재설계

AI가 잦은 실수를 저지른다면 프롬프트를 고칠 게 아니라, 헷갈리기 쉬운 스키마의 구조 자체를 명확하게 재설계하여 실수가 일어날 여지 자체를 없애버려야 합니다.

한 줄 정리: *스키마는 데이터 형식을 정한 계약서입니다. AI·개발자·시스템이 같은 형식으로 주고받게 해 자동화 오류를 줄입니다.*

참고 용어 사전 (Glossary)

업무자동화에 필요한 AI 실무 핵심 용어 한눈에 보기

Table 1. AI 작업에서 자주 쓰는 용어 20

🧠 모델·구조

토큰 Token
인공지능이 텍스트를 인식하고 처리하는 가장 작은 조각 단위 (예: 한글 한 글자, 또는 영어 한 단어가 하나의 토큰이 될 수 있음)

파라미터 Parameter
AI 모델의 지능 수준을 결정짓는 뇌세포(시냅스)의 수. 방대한 데이터 학습을 통해 형성된 가중치의 집합을 의미함.

임베딩 Embedding
사람이 쓰는 단어나 문장을 인공지능이 계산할 수 있도록, 의미와 문맥을 담은 숫자들의 배열(벡터)로 변환하는 과정

컨텍스트 윈도우 Context
AI 모델이 한 번의 대화에서 동시에 기억하고 처리할 수 있는 입력 및 출력 토큰의 최대 허용 한도

트랜스포머 Transformer
데이터의 순서와 관계없이 문장 내 핵심 단어들에 주목(Attention)하여 문맥을 이해하는 최신 대규모 언어 모델의 핵심 알고리즘 구조

🎯 입출력·제어

프롬프트 Prompt
원하는 결과를 얻기 위해 AI에게 입력하는 구체적인 질문, 명령, 또는 지시 사항

스키마 Schema
출력이 따라야 할 구조 명세

템퍼러처 Temperature
답의 무작위성 (↑과감 ↓안정)

멀티모달 Multimodal
텍스트+이미지+음성 함께 처리

토큰 비용 Token Cost
사용량(토큰) 단위 과금

📚 학습·지식

사전학습 Pretraining
방대한 글로 기본기 학습

파인튜닝 Fine-tuning
특정 용도로 추가 학습

RLHF
사람 피드백 기반 강화학습

RAG
외부 문서 검색해 근거로 답변

환각 Hallucination
그럴듯하게 틀린 답 생성

🤖 운영·에이전트

추론 Inference
학습된 모델이 답을 만드는 과정

에이전트 Agent
스스로 도구 쓰며 일 완수

MCP
AI에 도구·데이터 연결 표준

하네스 Harness
계획·도구·검증을 묶은 실행 틀

롤백 Rollback
직전 안전 상태로 되돌리기

활용. *모르는 용어가 나오면 이 표로 돌아오세요. 각 용어는 앞 슬라이드의 그림·예시와 1:1로 연결됩니다.*

실습 Codex·Claude 입력용 프롬프트 ①

공통 업무 자동화 UI를 만드는 실제 첫 프롬프트

↓ 전체 프롬프트 TXT

```
# 역할
당신은 업무 자동화 제품을 설계하고 구현하는 시니어 풀스택 개발자입니다.

# 목표
파일 기반 반복 업무를 자동화하는 공통 웹 UI를 만드세요.
이 UI는 이후 여러 업무 기능을 교체·추가할 수 있는 기본 템플릿이어야 합니다.

# 필수 화면
1. 업무현황 대시보드
2. AI 업무 설계
3. 파일 업로드
4. 처리 파이프라인
5. 자료·품목 매칭
6. 결과 미리보기와 작업 검증
7. Excel·PDF 다운로드
8. 작업 히스토리와 상세 로그

# 기본 원칙
- 처리 단계와 현재 상태를 항상 표시합니다.
- AI가 확신하지 못한 결과는 사람이 검토할 수 있어야 합니다.
- 원본 데이터는 보존하고 결과와 나란히 비교할 수 있어야 합니다.
- 외부 발송·삭제·확정은 사람의 승인 후 실행합니다.
```

실행 전 AI에게 요구할 것

① 바로 코딩하지 않기

② 필요한 질문 최대 3개

③ 화면 구조 먼저 제안

④ 샘플 데이터로 동작 확인

⑤ 완료 조건과 테스트 제시

> Codex, Claude Code 등 프로젝트 파일을 직접 수정할 수 있는 코딩 에이전트의 작업 폴더에서 실행합니다.

실습 Codex·Claude 입력용 프롬프트 ②

프롬프트 마지막에 구현 순서와 완료 조건을 붙인다

↓ TXT 다운로드

```
# 작업 방식
1. 현재 작업 폴더의 파일과 기술 구성을 먼저 확인하세요.
2. 구현 전에 화면 구조, 데이터 흐름, 필요한 라이브러리와 위험 요소를 설명하세요.
3. 정보가 부족하면 임의로 결정하지 말고 한 번에 최대 3개씩 질문하세요.
4. 질문에 답하기 어렵다면 추천 기본값과 그 이유를 제시하세요.
5. 공통 UI를 먼저 만들고, 실제 업무 기능은 작은 단계로 나누어 추가하세요.
6. 각 단계가 끝날 때 정상·경계·실패 사례를 테스트하세요.
7. 기존 정상 기능을 유지하며 최소 범위만 수정하세요.

# 첫 번째 결과물
아직 실제 업무 처리 기능은 구현하지 마세요.
먼저 실행 가능한 공통 UI, 메뉴 이동, 샘플 대시보드, 업로드 영역,
파이프라인 상태, 검증 목록, 히스토리와 로그 화면까지만 완성하세요.

# 완료 조건
- 프로젝트 실행 방법을 README에 기록합니다.
- 오류 없이 로컬에서 실행됩니다.
- 모든 메뉴와 화면을 직접 확인할 수 있습니다.
- 샘플 데이터로 정상·검토 필요·실패 상태가 모두 표시됩니다.
- 구현 결과와 다음 기능 추가 순서를 보고합니다.
```

왜 두 번에 나누나. *첫 요청에서는 재사용할 그릇만 만들고, 다음 요청부터 업로드·매칭·검증 같은 실제 기능을 하나씩 연결합니다.*

작성 내 업무 자동화 기획 프롬프트

빈칸만 채우면 AI가 요구사항과 구현 순서를 만든다

↓ 작성용 TXT

```
# 내 업무
업무명: [_____]
업무 목적: [_____]
수행 주기: [매일 / 매주 / 매월 / 수시]

입력 자료: [Excel / PDF / 이메일 / DB / 기타]
반복 작업: [_____]
사람의 판단: [_____]
자주 발생하는 오류: [_____]
최종 결과물: [Excel / PDF / 알림 / 시스템 입력]
보고·전달 대상: [_____]

반드시 지킬 규칙: [_____]
사람이 승인할 단계: [_____]
자동화하면 안 되는 일: [_____]
```

```
# AI에게 요청
위 업무를 수집·정리·판단·검증·보고 단계로 분해하세요.

자동화 후보를 다음 기준으로 비교하세요.
- 예상 절감 시간
- 구현 난이도
- 필요한 데이터와 도구
- 사람이 검토해야 할 부분
- 예상 결과물

바로 구현하지 말고, 완성도에 영향을 주는 질문을 한 번에 최대 3개씩 하세요.
모르는 항목에는 추천 기본값과 이유를 제시하세요.

답변이 끝나면 다음을 만들어 주세요.
1. 현재 업무 흐름
2. 개선된 자동화 흐름
3. 화면과 기능 목록
4. 데이터 구조와 처리 규칙
5. 예외·승인·보안 기준
6. 구현 우선순위
7. 완료 조건과 테스트 사례
8. 공통 UI에 기능을 추가할 최종 구현 프롬프트
```

실제 사례 1 모바일판을 네 번 다시 만든 이유

결과물의 이름보다 사용 환경과 완료 기준을 말해야 합니다

처음 요청

모바일 버전으로 볼 수 있게 만들어줘.

- PC 화면을 그대로 축소함
- 확대해야 읽을 수 있음
- 카드가 어색하게 잘림
- 워터마크가 본문과 겹침

개선 요청

```
모바일판은 1080×1920의 9:16 세로형으로 만들어줘.
PC 슬라이드를 축소하거나 좌우로 자르지 말고,
제목·본문·UI를 의미 단위 카드로 다시 배치해줘.
확대 없이 읽혀야 하고 본문은 최소 30px로 해줘.
워터마크는 콘텐츠 밖 하단 여백에 배치해줘.
```

> **배운 기준.** "모바일"이라는 이름만으로는 부족합니다. 방향·해상도·최소 글자·분할 방식·완료 상태를 함께 적어야 합니다.

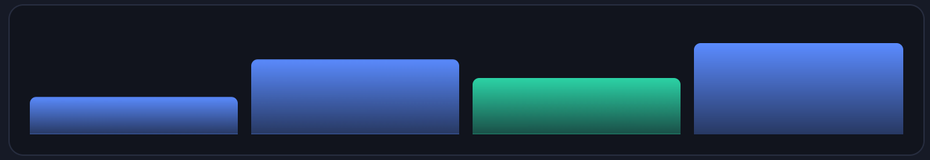
실제 사례 2 업무 UI가 통계 화면이 된 이유

필요한 기능과 함께 만들지 않을 것도 알려줍니다

AI가 흔히 떠올리는 화면

72%
자동화율

128
처리 건수



보기에는 그럴듯하지만 실제 업무를 시작할 수 없습니다.

실제로 필요했던 업무 행동

- 파일 업로드
- 품목·자료 매칭
- 미리보기
- 원본 비교 검증
- Excel·PDF
- 이력·로그

통계 대시보드가 아니라 업무를 실행하는 화면을 만들어줘.
요청하지 않은 통계와 차트는 추가하지 마.

> **배운 기준.** 화면 구성요소가 아니라 사용자가 실제로 해야 할 행동을 순서대로 설명합니다.

실제 기록 44개 Git 커밋에서 찾은 반복 패턴

빠르게 고치는 것보다 중요한 것은 같은 문제를 다시 만들지 않는 기준입니다

44
전체 커밋

26
15분 안에 이어진 수정

15,431
추가된 줄

5,699
삭제된 줄



> **숫자의 의미.** 커밋이 많은 것이 문제는 아닙니다. 구조 없이 빠르게 수정하면 개선 속도만큼 재작업도 빨라집니다.

재작업 방지 새 요청 전에 하는 한 가지 판단

새 내용을 만들기 전에 추가·대체·통합부터 정합니다

추가

기존 학습 자료에 없는 새로운 주장이나 실습을 넣습니다.

예: 루프와 오케스트레이션

대체

효과가 낮은 설명을 더 구체적인 사례로 바꿉니다.

예: 텍스트 카드 → 실제 대화

통합

같은 주장을 반복하는 여러 장을 하나로 합칩니다.

예: 역질문·역제안 중복 정리

```
먼저 기존 슬라이드에서 겹치는 내용을 찾아줘.
이번 요청이 추가·대체·통합 중 무엇인지 추천하고,
들어갈 위치와 없어져도 되는 내용을 보여준 뒤 수정해줘.
```

> **핵심.** AI에게 "무엇을 더할지"만 말하지 말고, 기존 결과물과의 관계를 판단하게 합니다.

작성 실습 AI에게 일을 맡기는 6칸 요청서

완벽한 문장 대신 결과를 결정하는 여섯 가지를 채웁니다

1. 목적
내가 만들고 싶은 것은 [ ]이다.

2. 사용자
이 결과물을 사용하는 사람은 [ ]이다.

3. 필수 기능
반드시 작동해야 하는 것은 [ ]이다.

4. 기존과의 관계
이번 작업은 추가 / 대체 / 통합이다.

5. 제외 범위
임의로 추가하지 않을 것은 [ ]이다.

6. 완료 조건
[ ] 상태가 되면 작업이 끝난다.

바로 만들지 말고 결과를 크게 바꿀 수 있는 질문을 최대 3개만 해줘.

각 질문에는 추천 기본값과 그 이유도 함께 알려줘.

> **바이브 코딩의 핵심.** 완벽한 첫 프롬프트가 아니라, 기준을 남기고 작은 결과를 확인하며 확장하는 것입니다.

한 장 정리 업무 자동화 완성 로드맵

결국 해야 할 일은 이 8단계입니다

01
업무 하나 선택
시간이 오래 걸리고 반복되는 일

02
현재 흐름 기록
입력·반복·판단·출력

03
AI에게 역제안
효과·난이도·우선순위

04
요구사항 확정
추가 질문·규칙·완료 조건

05
공통 UI 생성
메뉴·업로드·파이프라인·로그

06
기능 하나씩 추가
업로드 → 처리 → 검증 → 보고

07
샘플로 검증
정상·경계·실패·롤백

08
작게 운영 시작
사람 승인 후 자동 범위 확대

> **최종 산출물.** 작동하는 UI + 실제 업무 기능 + 검증 화면 + Excel·PDF + 히스토리·로그 + 다음 기능을 추가할 수 있는 요구사항 명세.

NEXT STEP 오늘 만든 기준을 다음 작업에도 씁니다

첫 과제는 작게, 기록은 다음에도 쓸 수 있게 남깁니다

1

업무 한 개

반복되는 업무를 5줄로 설명합니다.

2

샘플 한 개

작은 입력으로 결과와 예외를 확인합니다.

3

기록 한 개

다음 행동과 기준을 CURRENT_TASK.md에 남깁니다.

↓ Codex 시작 가이드

첫 MD, Skill, 설정, 효율을 실습 순서로 다시 봅니다.

↓ Astra 강의 가이드

설문, 4주 과정, 명령어와 실습 프롬프트를 확인합니다.

> **자료는 보조 도구입니다.** 실제 사용 가능 기능과 결과는 내 계정·입력·검증 기준으로 다시 확인합니다.

결론 핵심 결론

결국, AI의 성능은 언어능력에서 갈리고, 능력은 역설계(逆設計)로 결정된다.

🎯 **원하는 결과**
"이 모습이 목표"

◀

🧩 **필요한 단계**
무엇이 있어야 하나

◀

🧰 **도구·데이터**
어떤 자원이 필요

◀

🗣️ **지시로 구현**
AI에 명확히 전달

◀ *결과에서 거꾸로 분해해 설계 = 역설계*

"많이 다뤄볼수록, 내가 원하는 걸 직접 만들어내는 힘이 커진다"

★ 학습 후 바로 적용

오늘 단 한 가지만: 내 반복 업무를 5줄로 적어보세요

프로그램을 완성하는 것이 오늘의 목표가 아닙니다. 자동화할 업무 하나를 정확히 발견하는 것이 시작입니다.

✍️ 10분 작성

```
나는 매주 [_______] 업무를 한다.
입력 자료는 [_______]이다.
반복 작업은 [_______]이다.
사람이 판단하는 부분은 [_______]이다.
최종 결과물은 [_______]이다.
```

🤖 10분 AI 대화

이 업무를 단계별로 분해해줘.

자동화 후보를 효과와 난이도로 비교해줘.

바로 만들지 말고 필요한 질문을 해줘.

마지막에 요구사항과 구현 순서를 작성해줘.

💡 **오늘 남겨야 할 결과: “내가 자동화할 첫 번째 업무” 한 장짜리 기획서**